

EVOKE AI CMO — Complete Project Documentation

Single-document edition. Consolidates the full documentation set: system overview, frontend, backend, n8n automation, integrations, data model, deployment, known gaps, and a complete file index.

Complete technical documentation for the **EVOKE AI CMO** platform: frontend, backend, and all third-party integrations.

Prepared as a handover document. Repository: `evoked-cmo-frontend` · Branch documented: `vasanth-updates` Code as of: **5 August 2026** (173 commits, 19 May 2026 → 3 Aug 2026)

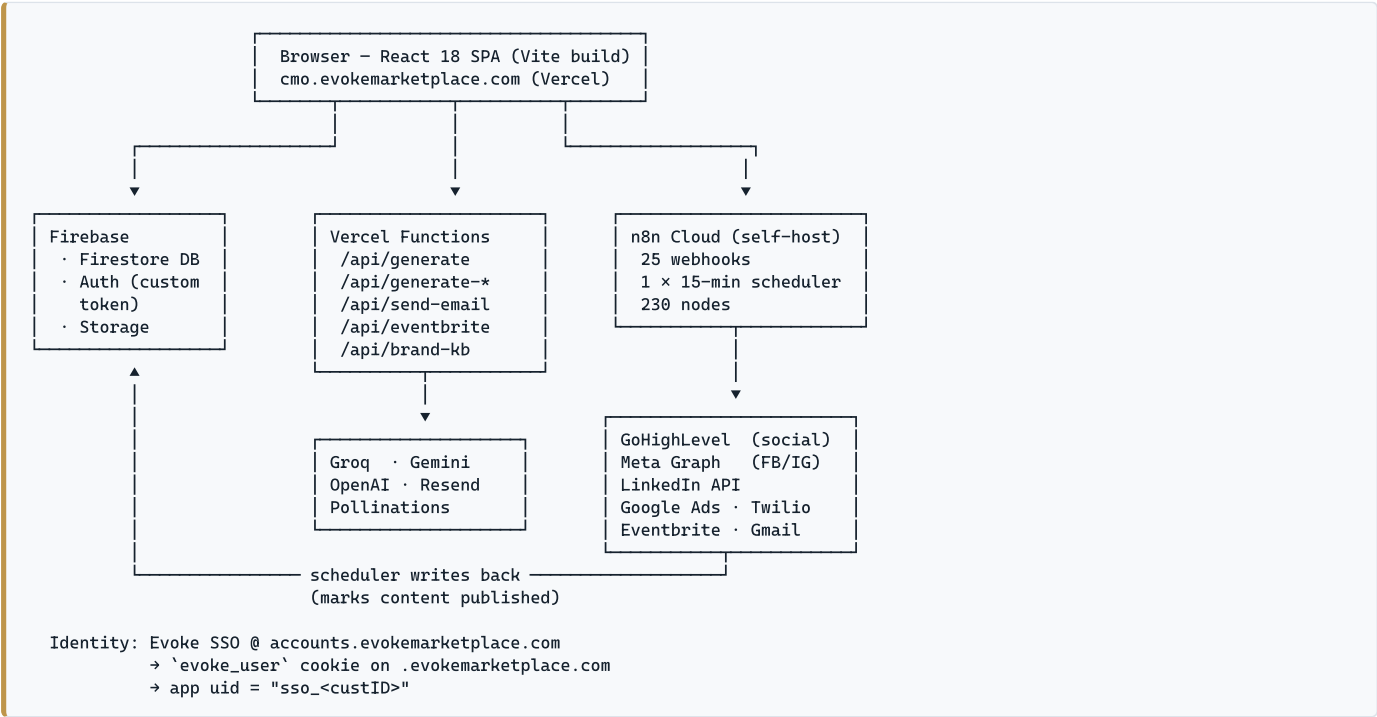
What this product is

EVOKE AI CMO is an **AI-powered Chief Marketing Officer** delivered as a web app. A user signs in with their Evoke Marketplace account, describes their brand once, and the platform then does the work a marketing department would do:

1. Builds a marketing **strategy** (annual / quarterly / monthly plans, SWOT, competitor intel)
2. Generates **campaigns** — social captions, blog posts, emails, ad copy, reel scripts
3. Generates **creative** — banners, product images, lifestyle photos, video
4. Routes everything through an **approval queue**
5. **Publishes** to the user's real Facebook / Instagram / LinkedIn / TikTok / Threads / Google Business accounts, plus WhatsApp, SMS and email
6. Reports back with **analytics** and an **executive report**

Roughly **100 routes**, **75 pages**, **20 components**, **14 service modules**, **7 backend API routes**, and one **230-node n8n workflow** carrying the publishing and OAuth work.

The 30-second architecture



Quick start for the next developer

```
npm install
```

Copy `.env` from the current maintainer (it is **not** in git — see [Part 7](#) for the complete list of required keys), then:

```
npm run dev
```

The app runs on **http://localhost:3007**.

On `localhost` the app auto-signs-in as a hard-coded dev profile and **all plan gates are bypassed**, so every page is reachable without a real Evoke SSO session. See [Part 1 § Local development bypasses](#).

Document conventions

- File paths are relative to the repository root.
- "n8n" always refers to the single hosted workflow `EVOKE Cmo` (file: `EVOKE-CMO-v21-GHL.json`).
- **No secret values appear anywhere in this documentation.** Only variable *names* are listed. Where secrets are currently hard-coded in source, that fact is flagged in [Part 8](#) without reproducing the value.

Table of Contents

Part 1 — System Overview

- 1. What the system does
- 2. Technology stack
- 3. Repository layout
- 4. Authentication — how a user gets in
- 5. The complete user journey, start to finish
- 6. Navigation / information architecture
- 7. Design system
- 8. Where each kind of work happens

Part 2 — Frontend Reference

- 1. Entry points
- 2. Global styling — `src/index.css`
- 3. Routing — `src/App.jsx`
- 4. Complete route table
- 5. Components (`src/components/`)
- 6. Hooks (`src/hooks/`)
- 7. Libraries (`src/lib/`)
- 8. Services (`src/services/`)
- 9. Pages (`src/pages/`)
- 10. Configuration modules

Part 3 — Backend Reference

- Layer 1 — Vercel Functions (`api/`)
- Layer 2 — Vite dev middleware (`vite.config.js`)
- Layer 3 — n8n

- Deployment configuration — `vercel.json`
- Request-flow summary

Part 4 — n8n Automation Backend

- 1. Complete webhook index
- 2. Social OAuth flows (6 webhooks)
- 3. The main campaign engine — `evoke-cmo`
- 4. The AI agents endpoint — `evoke-agents`
- 5. GoHighLevel integration (7 webhooks)
- 6. Paid ads (5 webhooks)
- 7. Messaging webhooks (3)
- 8. The scheduled publisher — the most important flow
- 9. Credential storage strategy
- 10. Operating the workflow

Part 5 — Third-Party Integrations

- Integration status at a glance
- 1. Evoke SSO — identity
- 2. Firebase — the primary data layer
- 3. GoHighLevel — the social publishing engine
- 4. Meta — Facebook, Instagram, WhatsApp, Ads
- 5. LinkedIn
- 6. Twilio — WhatsApp & SMS
- 7. AI providers
- 8. Google Ads
- 9. Eventbrite
- 10. TikTok
- 11. Email
- 12. Payments
- 13. Supabase — built, then rolled back
- 14. EGT — Evoke Gratitude Token (blockchain)
- 15. Analytics & media
- 16. Not integrated — but presented in the UI

Part 6 — Data Model

- Firestore collection map
- `users/{uid}` — the central document
- `content_items/{itemId}` — generated content
- `users/{uid}/contacts/{contactId}` — CRM
- `users/{uid}/teamMembers/{memberId}` — Team
- `users/{uid}/teamInvites/{inviteId}` — Invitations
- `users/{uid}/inboxThreads/{threadId}` — Social Inbox replies
- `governance_audit_log/{entryId}` — Brand Governance audit trail
- Firestore security rules
- Supabase Postgres schema (not live)

Part 7 — Environment & Deployment

- 1. Prerequisites
- 2. Local development
- 3. Environment variables

- 4. Deployment — frontend (Vercel)
- 5. Deployment — n8n
- 6. Firebase setup
- 7. Post-deployment verification checklist
- 8. Troubleshooting

Part 8 — Known Gaps & Risks

- Severity summary
-  High severity
-  Medium severity
-  Low severity
- Features presented in the UI without a real data source
- Architectural constraints worth understanding before changing anything
- Recommended priority order for the next developer
- What is genuinely working well

Part 9 — Complete File Index

- Root
 - `api/` — Backend (Vercel functions)
 - `src/` — Entry points & config
 - `src/components/` — 20 shared components
 - `src/hooks/` — 4 hooks
 - `src/lib/` — 12 logic modules
 - `src/services/` — 14 data-access modules
 - `src/pages/` — 75 pages
 - `public/` — Static assets
 - `docs-internal/` — Internal specs (git-ignored)
 - `docs/` — This documentation
 - External artifacts (not in the repository)
 - Handover checklist
-

Part 1 — System Overview

1. What the system does

EVOKE AI CMO replaces the function of a marketing team with a chain of AI agents. The user provides brand context once; every downstream feature reads from that context, so campaigns, creative and strategy all stay on-brand without the user re-typing anything.

The platform is sold in four tiers, and features are gated by tier:

Plan key	Product name	Price	Broad capability
<code>free</code>	Free Trial	\$0 (14-day trial)	Brand setup, strategy, health score, connect accounts, publish
<code>package-a</code>	Starter	\$30/mo	+ Content generation, captions, reels, copywriting, images, video
<code>package-b</code>	Professional	\$100/mo	+ AI agents (SEO, Email, A/B), CRM, analytics, social manager, approval queue
<code>package-c</code>	Enterprise	Custom	+ Paid ads (Meta/Google), multi-channel execution, team, C-suite agents

Tiers are **cumulative** — Package B includes everything in A. Source of truth: `src/lib/planGate.js`.

2. Technology stack

Frontend

Layer	Choice	Version	Notes
Framework	React	18.2	Function components + hooks only, no class components
Build tool	Vite	5.1	Also hosts the dev-time API layer via a custom plugin
Routing	react-router-dom	6.22	<code>BrowserRouter</code> , v7 future flags enabled
Icons	lucide-react	0.323	Used consistently across every page
Animation	framer-motion	11.0	Page transitions, modals, plan-gate card
HTTP	native <code>fetch</code> + axios	1.6	<code>fetch</code> dominates; axios present as a dependency
Styling	Inline styles + one global CSS file	—	No Tailwind, no CSS-in-JS library, no CSS modules

Backend

Layer	Choice	Notes
Hosting	Vercel	Static SPA + serverless/edge functions
API functions	Vercel Edge Runtime (6 routes) + Node runtime (1 route)	<code>api/</code> directory
Automation engine	n8n (self-hosted at <code>n8n-zvxi.srv1837606.hstgr.cloud</code>)	230 nodes, 25 webhooks, 1 scheduler
Database (primary)	Firebase Firestore	All live application data
Database (secondary)	Supabase Postgres	Schema built, partially wired, currently not in the live path
Auth	Evoke SSO + Firebase Auth custom tokens	See §4
File storage	Firebase Storage + Cloudinary + ImgBB	Images land in different places depending on the flow

AI providers

Provider	Model	Used for	Called from
Groq	llama-3.1-8b-instant	Campaign copy packages	api/generate-campaign.js
Groq	llama-3.3-70b-versatile	Specialist text agents (SEO, Reddit, X, LinkedIn, GEO...)	Vite dev /api/run-agent
Groq	caller-specified	Generic chat completions proxy	api/generate.js
Google Gemini	gemini-2.0-flash	Campaign content + agent responses inside n8n	n8n HTTP nodes
Google Gemini	gemini-2.0-flash-exp	Image generation	api/generate-banner.js , api/gemini-image.js
OpenAI	gpt-image-1 / dall-e-3	Optional image generation provider	api/generate-banner.js
Pollinations	flux	Free fallback image generation, no API key	api/generate-banner.js

3. Repository layout

evoke-cmo-frontend/	
├─ api/	← Backend: Vercel serverless & edge functions
│ └─ _lib/	← Shared server-only helpers (Supabase admin, auth verify)
│ └─ brand-kb.js	← Node runtime (needs firebase-admin)
│ └─ eventbrite.js	← Edge
│ └─ gemini-image.js	← Edge
│ └─ generate-banner.js	← Edge
│ └─ generate-campaign.js	← Edge
│ └─ generate.js	← Edge
│ └─ send-email.js	← Edge
├─ docs/	← THIS DOCUMENTATION
├─ docs-internal/	← Legacy .docx specs + supabase-schema.sql
├─ public/	← Static assets (logo, favicons, screenshots, platform verification files)
├─ src/	
│ └─ components/	← 20 shared UI components
│ └─ hooks/	← 4 React hooks
│ └─ lib/	← 12 pure-logic helper modules
│ └─ pages/	← 75 route-level page components
│ └─ services/	← 14 data-access / API-client modules
│ └─ App.jsx	← Router + all ~100 route definitions
│ └─ config.js	← n8n webhook URL resolution
│ └─ firebase.js	← Firebase SDK initialisation
│ └─ index.css	← The ONLY global stylesheet (347 lines)
│ └─ main.jsx	← React entry point
├─ index.html	← HTML shell + Google Ads gtag
├─ package.json	
├─ vercel.json	← SPA rewrite rule
└─ vite.config.js	← Build config + dev-only API middleware + CORS proxies

4. Authentication — how a user gets in

This is the single most important thing to understand, because **the app does not have its own login**. It is a satellite of the Evoke Marketplace identity system.

The production flow

1. User hits any page on `cmo.evokemarketplace.com`
2. `useEvokeSession()` reads the `'evoke_user'` cookie
(written by `accounts.evokemarketplace.com` at the PARENT domain
`'evokemarketplace.com'`, so every subdomain can read it)
3. No cookie? → `fetchSessionFromBackend()` calls
`GET {VITE_API_BASE_URL}/auth/session` with `credentials:'include'`
to recover a session from the `httpOnly 'auth_token'` cookie
4. Still nothing? → `status = 'unauthenticated'`
`redirectToLogin()` sends the browser to
`accounts.evokemarketplace.com/login/?redirect_url=<current page>`
5. Accounts portal authenticates, writes `'evoke_user'`, redirects back
6. `profileToUser()` maps the SSO profile to the app's user object:
`uid = "sso_" + profile.custID` ← THE APP'S USER ID
`displayName, email, token, custID ...`
7. `AuthProvider` puts `{ profile, status, user }` on React context
→ every page reads it via `useAuth()`

Key file map:

Step	File
Cookie read/write/parse, JWT decode, sign-out	<code>src/lib/session.js</code>
React subscription to the cookie	<code>src/hooks/useEvokeSession.js</code>
SSO profile → app user mapping	<code>src/lib/authUtils.js</code>
Context provider	<code>src/components/AuthProvider.jsx</code>
Consumer hook	<code>src/hooks/useAuth.js</code>

The secondary Firebase path

`App.jsx` also contains `EvokeAuthHandler`. If the app is loaded with a `?token=` / `?customToken=` / `?access_token=` / `?auth_token=` query parameter, it:

1. Strips the token from the URL (`history.replaceState`)
2. Calls `signInWithCustomToken(auth, token)` — establishing a **real Firebase Auth session**
3. Calls `getOrCreateUser()` to create the Firestore user document
4. Navigates to `sessionStorage.evoke_post_login_route` (default `/dashboard`)

Why both exist: the `evoke_user` cookie identifies the user for the UI, but Firestore security rules and the Node API route (`api/brand-kb.js`) need a *verifiable* identity. The Firebase custom token is minted with the same `sso_<custID>` uid, so a verified Firebase ID token yields exactly the uid the rest of the app already uses — no second identity system. This reasoning is documented in `api/_lib/verifyUser.js`.

Local development bypasses

Three separate dev bypasses exist. Know them, because they hide real production behaviour:

Bypass	File	Effect on <code>localhost</code> / <code>127.0.0.1</code>
Auto sign-in	<code>src/hooks/useEvokeSession.js</code>	Falls back to a hard-coded <code>DEV_PROFILE</code> (<code>custID: 260417001</code>) with <code>token: null</code> . You are always "logged in".
Plan gates off	<code>src/components/PlanGate.jsx</code>	<code>IS_DEV</code> short-circuits — every gated page renders regardless of plan.
API auth off	<code>api/_lib/verifyUser.js</code>	When <code>NODE_ENV !== 'production'</code> , trusts <code>req.body.uid</code> / <code>req.query.uid</code> with no token check.
Cookie expiry off	<code>src/lib/session.js</code>	JWT expiry checks skipped; cookie forced to 7 days.

Bypass	File	Effect on <code>localhost</code> / <code>127.0.0.1</code>
Session recovery off	<code>src/lib/session.js</code>	<code>fetchSessionFromBackend()</code> returns early — the auth cookie is domain-locked to <code>.evokemarketplace.com</code> and would only produce noisy 401s.

`/dev-reset` and `/supabase-test` routes are also **localhost-only** — in production they redirect to `/`.

5. The complete user journey, start to finish

This is the "how it works from starting to ending" walkthrough.

Stage 0 — Landing

`src/pages/Landing.jsx` (4,392 lines) is the public marketing page. No auth required. CTAs push the visitor toward `/pricing` or sign-in.

Stage 1 — Brand Setup

Route: `/brand-profile` → `src/pages/BrandProfilePage.jsx`, which renders `src/components/OnboardingWizard.jsx` (588 lines).

Six wizard steps:

#	Step label	Collects
1	Cognitive System Tuning	Which AI agents to enable (Strategy, Content, Creative, SEO...)
2	Brand System Definition	Industry, company size, brand voice, value tags
3	Connect System API	10 integration toggles (GA, Meta Ads, Instagram, LinkedIn, TikTok, Gmail, HubSpot, Salesforce, Shopify, WordPress)
4	Marketing Plan Tuning	Primary goal — revenue / awareness / leads
5	Neural Network Ingestion	Document upload for brand context
6	Cognitive Audit Report	Summary + plan selection

On finish it writes to Firestore: `knowledgeBase` (via `saveKnowledgeBase`), `selectedPlan` / `userPlan`, `brandSetupComplete`, and `recommendedRoutes`.

`/onboarding` and `/setup` are **legacy routes that now redirect here** — the old chat-wizard chain was retired. This is the only onboarding flow the rest of the app reads from.

Stage 2 — Dashboard

Route: `/dashboard` → `src/pages/DashboardPage.jsx`.

Reads the Brand Knowledge Base and feeds it to `src/lib/recommendations.jsx` (`getRecommendedActions`), which scores a pool of suggested next actions against the user's industry, audience type and objectives, then renders the top ones as cards. Recommendations are industry-aware: e-commerce brands get product-campaign and visual suggestions; B2B/SaaS brands get email-drip and LinkedIn suggestions.

`src/components/ProductTour.jsx` runs a first-visit guided tour, anchored to `data-tour` attributes on the sidebar nav items. Dismissal is persisted with `markTourSeen()`.

Stage 3 — Connect Channels

Route: `/connect-accounts` → `src/pages/ConnectAccounts.jsx` (2,273 lines).

Two connection mechanisms coexist:

(a) Via GoHighLevel — the current live path for social publishing `src/services/ghlService.js` drives it: 1. `ensureGhlLocation(uid)` — provisions a GHL sub-account ("location"), or falls back to a shared workspace ID 2. `startSocialConnect(uid, platform)` — asks n8n for an OAuth start URL, opens a popup, waits for a `postMessage` from a **trusted origin allow-list** (`services.leadconnectorhq.com`, `app.gohighlevel.com`) 3. `listConnectedPages()` — lists pages the sign-in authorised. Returns `scoped: true` only if the list could be narrowed to *this* sign-in. **If it can't, the UI must not auto-select a page** — on a shared workspace that would silently link someone else's page. 4. `attachSocialPage(uid, platform, accountId, page)` — records the chosen page under `socialAccounts.<platform>` in Firestore

(b) Direct OAuth via n8n — for Facebook, Instagram, LinkedIn, Gmail, Eventbrite, TikTok, Google Ads and Meta Ads. Each has a dedicated n8n webhook that exchanges the auth code for tokens server-side and returns a normalised profile. See [Part 4](#).

Stage 4 — Strategy

Routes: `/strategy-home`, `/strategy`, `/competitor-intel`, `/swot-analysis`.

`MarketingStrategyPage.jsx` calls the n8n agent webhook (or `/api/run-agent` in dev) and produces annual/quarterly/monthly plans plus campaign recommendations. The output is written back to the Brand Knowledge Base via `appendJourneyOutput(oid, 'strategy', summary)` so later steps have it as context.

Stage 5 — Campaign creation

Routes: `/campaigns`, `/new-campaign`, `/campaign/:type`, `/campaign-hub`.

`src/pages/CampaignForm.jsx` is the largest file in the codebase (**4,488 lines**) and the heart of the product. It:

1. Collects campaign inputs (type, name, description, goal, audience, dates, platforms)
2. Posts to the n8n `/webhook/evoke-cmo` endpoint (`WEBHOOK_URL` in `src/config.js`)
3. n8n responds immediately (acknowledgement), then generates content with Gemini and publishes to whichever platforms have credentials attached
4. Saves the generated items to Firestore `content_items` via `saveContentItems()`
5. For multi-day campaigns, writes days 2..N via `scheduleCampaignDays()` with `status: 'scheduled'` and `requiresApproval: false`

Why `scheduleCampaignDays` matters: before it existed, days 2..N were posted with a browser `setTimeout`, which meant closing the tab cancelled the campaign. Now the rows live in Firestore and the n8n 15-minute scheduler picks them up headlessly.

Handoff between journey steps is defined in `src/lib/journeyHandoff.js`, which encodes the canonical **12-step CMO journey** and per-step mappers that shape one agent's output into the next agent's pre-fill payload.

Stage 6 — Content & creative generation

Route	Page	Produces
<code>/caption-suite</code>	CaptionSuitePage.jsx	Platform-specific social captions
<code>/reel-scripts</code>	ReelScriptsPage.jsx	Short-form video scripts
<code>/blog-generator</code>	BlogGeneratorPage.jsx	Long-form blog posts
<code>/email-composer</code>	EmailComposerPage.jsx	Email campaigns
<code>/content-gen</code>	ContentGenerationPage.jsx	Newsletters & general content
<code>/copywriting</code>	CopywritingAgentPage.jsx	Ad and landing-page copy
<code>/product-desc</code>	ProductDescription.jsx	Product descriptions
<code>/image-generator</code>	AllImageGeneratorPage.jsx	AI images
<code>/image-angles</code> , <code>/image-360</code> , <code>/image-seo</code> , <code>/image-lifestyle</code> , <code>/image-3d</code>	ImageToolPage.jsx	Product photography variants (one component, five routes)
<code>/creative-asset</code>	CreativeAssetPage.jsx	Banners & ad creatives
<code>/video-gen</code>	VideoGenerationPage.jsx	Video generation

`src/services/bannerService.js` (353 lines) handles banner composition — it can build an event poster on an HTML `<canvas>` locally, or call `/api/generate-banner` for a model-generated image.

Stage 7 — Governance & approval

- `/brand-governance` → `BrandGovernancePage.jsx` — conformance review, audit trail persisted by `src/services/governanceService.js`
- `/compliance-agent` → `CompliancePage.jsx`

- `/queue` → `ApprovalQueue.js` — reads `content_items` where `status = 'draft'`. Approving sets `status: 'approved'`; scheduling calls `scheduleItem()` which sets `status: 'scheduled'` and `requiresApproval: false` (a human already approved it).

Stage 8 — Publishing

Two paths:

Immediate publish — `/post-content` → `PostContent.jsx` (1,307 lines) → `publishSocialPost()` in `ghlService.js` → n8n `/webhook/evoke-social-post` → GoHighLevel Social Planner API → the user's real accounts.

Scheduled publish — the n8n **Schedule Trigger** fires every 15 minutes:

```

Schedule Trigger (every 15 min)
├─ Query Due Content      → Firestore runQuery on content_items
├─ Has Due Items?        → IF
├─ Split Due Items
├─ Map Firestore Doc to Day Payload
├─ Day Normalize & Flatten
│   ├── Day Has LinkedIn? → LinkedIn ugcPosts API
│   ├── Day Has Instagram? → Graph API /media → /media_publish
│   ├── Day Has Facebook? → Graph API /photos
│   ├── Day Has WhatsApp? → Twilio Messages API
│   └─ Day Has SMS?       → Twilio Messages API
├─ Merge Publish Results
└─ Mark Content Published → Firestore PATCH
  
```

The final node writes back to Firestore so the item is not published twice.

Stage 9 — Analytics & reporting

- `/analytics` → `AnalyticsDashboard.jsx`
- `/campaign-performance/:campaignId` → `CampaignPerformancePage.jsx`
- `/executive-report` → `ExecutiveReportPage.jsx`
- `/marketing-attribution` → `MarketingAttributionPage.jsx`
- `/health-score` → `MarketingHealthPage.jsx`

Stage 10 — Paid ads (Package C)

- `/meta-ads-boost` → `MetaAdsBoost.jsx` → `metaAdsService.js` → n8n `/webhook/meta-ads-boost` → Meta Marketing API (creates campaign → ad set → creative → ad, in that order)
- `/hub/ads` → `AdsCenterHubPage.jsx` → `googleAdsService.js` → n8n `/webhook/google-ads-create-campaign` → Google Ads API v23 (budget → campaign → ad group → keywords → ad)

6. Navigation / information architecture

Defined in `src/components/AppSidebar.jsx`. Three groups, each filtered by the user's plan:

Primary (follows the product journey diagram's stage order) Dashboard → Connect Channels → AI Agents Hub → Campaigns → Campaign Execution → Strategy → Content Studio → Creative Studio → Video Studio → Analytics → Optimization

Growth Stack SEO Center · Ads Center · Social Media · Social Calendar · CRM · Audience · Automation

Utility (pinned to the bottom) Reports · Asset Library · Team · Billing · Settings

Sidebar behaviour worth knowing: - Collapsible: 260px expanded / 64px collapsed. The width is broadcast as the CSS custom property `--evox-sidebar-w` on `documentElement`, so any page can adapt its layout. - Sub-items only appear when their parent section is the active page. - Items above the user's plan are **hidden entirely**, not shown locked.

7. Design system

There is **no CSS framework**. Styling is:

1. `src/index.css` — 347 lines, defines CSS custom properties and base/reset styles

2. **Inline `style={{}}` objects** on every component — this is the dominant pattern

Two colour systems coexist in the codebase, a legacy one and the current Figma one:

Token	Legacy (<code>index.css</code>)	Current (sidebar / wizard)
Background	<code>--bg: #070b17 / #0e0c09</code>	<code>#0A0A0F</code>
Card	<code>--bg-card: #1c1a13</code>	<code>#111118</code>
Border	<code>rgba(255,255,255,0.06)</code>	<code>#2A2A3A</code>
Gold accent	<code>--gold: #c8973e</code>	<code>#BE954A</code>
Text	<code>--text: #f0ebe0</code>	<code>#FFFFFF</code>
Muted text	<code>rgba(240,235,224,0.55)</code>	<code>#9B9BB0</code>

Fonts (loaded from Google Fonts in `index.html` and `index.css`): `Inter` (body), `Syne` (legacy headings), `Outfit` (current headings), `Geist` (labels).

8. Where each kind of work happens

A quick decision table for the next developer:

I need to change...	Go to
A route or its plan gate	<code>src/App.jsx</code> + <code>src/lib/planGate.js</code>
What appears in the sidebar	<code>src/components/AppSidebar.jsx</code>
Which plan unlocks a feature	<code>src/lib/planGate.js</code> (<code>FEATURE_PLAN</code> + <code>ROUTE_PLAN</code>)
How a user's data is read/written	<code>src/services/*.js</code>
An AI prompt for a campaign	<code>api/generate-campaign.js</code> and <code>vite.config.js</code> (duplicated)
An AI prompt for a specialist agent	<code>vite.config.js</code> / <code>api/run-agent</code> <code>PROMPTS</code> object
How content is published to social	The n8n workflow — not this repo
OAuth token exchange for any platform	The n8n workflow — not this repo
Environment configuration	<code>.env</code> (local) / Vercel dashboard (production)

⚠ Prompt duplication: the campaign prompt exists in two places — `api/generate-campaign.js` (production) and `vite.config.js` (dev). They are currently identical and must be edited together. Flagged in [Part 8](#).

Part 2 — Frontend Reference

Complete reference for everything under `src/` — **54,561 lines** across 118 files.

1. Entry points

`index.html` (repo root)

The HTML shell Vite builds against.

- Sets `<meta name="color-scheme" content="dark">` — the app is dark-mode only.
- Preconnects to Google Fonts and loads **Inter** (weights 300–900).
- Injects the **Google Ads gtag** tracking script (conversion ID `AW-18246572299`).
- Mounts `<div id="root">` and loads `/src/main.jsx` as an ES module.

`src/main.jsx`

Ten lines. Creates the React 18 root, wraps `<App />` in `<React.StrictMode>`, imports `./index.css`. Nothing else happens here.

2. Global styling — `src/index.css`

347 lines. The only stylesheet in the project. Everything else is inline styles.

Structure:

Section	Contents
<code>@import</code>	Google Fonts — Syne (700/800) + Inter (300–900)
Reset	<code>*, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0 }</code>
<code>:root</code> custom properties	Colour, radius and transition tokens (see table below)
Base elements	<code>html</code> (smooth scroll, <code>overflow-x: clip</code>), <code>body</code> (Inter, dark bg, antialiased)
Scrollbar	5px wide, gold thumb (<code>rgba(200,151,62,0.3)</code>), darker track
Selection	<code>::selection { background: rgba(200,151,62,0.2) }</code>
Utility classes	Layout, card, button and animation helpers used across pages

CSS custom properties defined in `:root`

```
/* Core backgrounds */
--bg: #070b17; --bg-alt: #0b0a07; --bg-card: #1c1a13;
--bg-surface: #211f17; --bg-hover: #252318;

/* Gold accent */
--gold: #c8973e; --gold-light: #d4a853;
--gold-dim: rgba(200,151,62,0.13); --gold-border: rgba(200,151,62,0.22);

/* Text ramp */
--text: #f0ebe0;
--text-2: rgba(240,235,224,0.55);
--text-3: rgba(240,235,224,0.32);
--text-4: rgba(240,235,224,0.16);

/* Borders */
--border: rgba(255,255,255,0.06); --border-warm: rgba(200,151,62,0.18);

/* Radii */
--r-sm: 8px; --r-md: 12px; --r-lg: 16px; --r-xl: 20px; --r-pill: 100px;

/* Transition */
--t: all 0.2s ease;
```

One runtime-set custom property

`--evox-sidebar-w` is set on `document.documentElement` by `AppSidebar.jsx` whenever the sidebar expands or collapses (260px ↔ 64px). Pages read it to offset their own content.

Note on the two palettes. `index.css` holds the original warm/gold palette (`#0e0c09` / `#c8973e`). The newer Figma-derived screens (sidebar, onboarding wizard, hub pages) use a cooler set (`#0A0A0F` / `#111118` / `#BE954A` / `#2A2A3A`) declared inline as local constants rather than as CSS variables. Both are live. Unifying them is listed in [Part 8](#).

3. Routing — `src/App.jsx`

237 lines. Responsibilities:

- 1. Imports all 75 page components statically (no lazy loading / code splitting).
- 2. Defines the `G(plan, name, Comp)` helper — wraps a page in `<PlanGate>` (see below).
- 3. Defines `IS_LOCAL` — used to hide dev-only routes in production.
- 4. Contains `EvokeAuthHandler` — the Firebase custom-token sign-in path (see [Part 1 § Authentication](#)).
- 5. Renders the component tree (see below).

The `G()` plan-gate helper:

```
function G(plan, name, Comp) {
  return <PlanGate requiredPlan={plan} featureName={name}><Comp /></PlanGate>
}
```

The rendered tree:

```
<BrowserRouter future={{ v7_startTransition, v7_relativeSplatPath }}>
  <AuthProvider>
    <Routes> ... ~100 routes ... </Routes>
    <Chatbot />           ← global, present on every page
  </AuthProvider>
</BrowserRouter>
```

Catch-all: `<Route path="*" element={<Navigate to="/" replace />} />`

4. Complete route table

Public / unauthenticated

Route	Component	Notes
<code>/</code>	<code>Landing.jsx</code>	Marketing home page
<code>/signin</code>	<code>SignIn.jsx</code>	Legacy — redirects to the Evoke accounts portal
<code>/privacy</code>	<code>Privacy.jsx</code>	Privacy policy
<code>/terms</code>	<code>Terms.jsx</code>	Terms of service

Redirects (retired routes kept alive so old links don't 404)

Route	Redirects to	Why retired
<code>/onboarding</code>	<code>/brand-profile</code>	Legacy chat-wizard chain superseded by <code>OnboardingWizard</code>
<code>/setup</code>	<code>/brand-profile</code>	Same
<code>/cmo</code>	<code>/dashboard</code>	Legacy CMO command centre
<code>/hub/:agent</code>	<code>/agents-hub</code>	Old function-grouped hub (retired 2026-07-27)

Route	Redirects to	Why retired
/strategy-hub	/strategy-home	Superseded by the sidebar-linked page (retired 2026-07-27)
/executive-reporting	/executive-report	Was a duplicate page on hardcoded mock data (retired 2026-07-17)

Free tier

Route	Component
/dashboard	DashboardPage.jsx
/strategy-home	StrategyHome.jsx
/strategy	MarketingStrategyPage.jsx
/competitor-intel	CompetitorIntelPage.jsx
/swot-analysis	SwotAnalysisPage.jsx
/content-studio	ContentStudioHubPage.jsx
/hub/creative	CreativeStudioHubPage.jsx
/agents-hub	AgentsHub.jsx
/agent/:type	CmoAgentOverviewPage.jsx
/brand-profile	BrandProfilePage.jsx
/health-score	MarketingHealthPage.jsx
/campaign/:type	CampaignForm.jsx
/results	Results.jsx
/tokens	Tokens.jsx
/purchase	Purchase.jsx
/products	ProductsPage.jsx
/eventbrite-post	EventbritePost.jsx

Plans & packages (always accessible, ungated)

[/plans](#) · [/pricing](#) · [/free-plan](#) · [/package-a](#) · [/package-b](#) · [/package-c](#)

Package A — Creative & Content

Route	Component	Gate label
/caption-suite	CaptionSuitePage.jsx	Caption Suite
/reel-scripts	ReelScriptsPage.jsx	Reel Scripts
/content-gen	ContentGenerationPage.jsx	Content Generation
/copywriting	CopywritingAgentPage.jsx	Copywriting Agent
/product-desc	ProductDescription.jsx	Product Description
/blog-generator	BlogGeneratorPage.jsx	Blog Generator
/email-composer	EmailComposerPage.jsx	Email Composer
/image-generator	AIImageGeneratorPage.jsx	AI Image Generator
/image-angles	ImageToolPage.jsx	Product Image Angles
/image-360	ImageToolPage.jsx	360° Product Video

Route	Component	Gate label
/image-seo	ImageToolPage.jsx	SEO Product Images
/image-lifestyle	ImageToolPage.jsx	Lifestyle Photos
/creative-asset	CreativeAssetPage.jsx	Creative Assets
/video-gen	VideoGenerationPage.jsx	Video Generation
/hub/video-studio	VideoStudioHubPage.jsx	Video Studio
/brand-kb	BrandKnowledgeBase.jsx	Brand Knowledge Base
/kpi-recommendations	KpiRecommendationsPage.jsx	KPI Recommendations

Note: `ImageToolPage.jsx` serves **five** routes. It switches behaviour off the pathname.

Package B — AI Agents + Social

Route	Component
/campaigns	CampaignsPage.jsx
/new-campaign	NewCampaignWizardPage.jsx
/campaign-performance/:campaignId	CampaignPerformancePage.jsx
/email-marketing	EmailMarketingPage.jsx
/seo-agent	SeoAgentPage.jsx
/hub/seo	SeoIntelligenceCenterPage.jsx
/ab-testing	ABTestingPage.jsx
/marketing-attribution	MarketingAttributionPage.jsx
/audience-builder	AudienceBuilder.jsx
/trends	TrendAnalysis.jsx
/crm	CrmPage.jsx
/analytics	AnalyticsDashboard.jsx
/executive-report	ExecutiveReportPage.jsx
/campaign-hub	CampaignHub.jsx
/connect-accounts	ConnectAccounts.jsx
/queue	ApprovalQueue.jsx
/post-content	PostContent.jsx
/inbox	SocialInbox.jsx
/brand-governance	BrandGovernancePage.jsx
/hub/social	SocialMediaManagerPage.jsx
/social-calendar	SocialCalendarPage.jsx
/image-3d	ImageToolPage.jsx

Package C — Paid Ads & Full Deploy

Route	Component
<code>/meta-ads-boost</code>	<code>MetaAdsBoost.jsx</code>
<code>/hub/ads</code>	<code>AdsCenterHubPage.jsx</code>
<code>/execution</code>	<code>MarketingExecutionPage.jsx</code>
<code>/team</code>	<code>TeamManagement.jsx</code>
<code>/partner-sharing</code>	<code>PartnerSharing.jsx</code>
<code>/compliance-agent</code>	<code>CompliancePage.jsx</code>
<code>/ai-cfo</code> <code>/ai-cto</code> <code>/ai-ceo</code> <code>/ai-cro</code>	<code>CSuitePage.jsx</code> (all four)

Localhost-only

Route	Component	Production behaviour
<code>/dev-reset</code>	<code>DevResetPage.jsx</code>	Redirects to <code>/</code>
<code>/supabase-test</code>	<code>SupabaseTestPage.jsx</code>	Redirects to <code>/</code>

⚠️ **Route/gate drift:** `App.jsx` gates `/image-3d` at `package-b`, but `ROUTE_PLAN` in `planGate.js` and `FEATURE_PLAN.image_3d` disagree with each other. `App.jsx` is what actually runs. See [Part 8](#).

5. Components (`src/components/`)

20 files.

Layout & navigation

File	Lines	Purpose
<code>AppSidebar.jsx</code>	372	The main app navigation. Three plan-filtered groups (Primary / Growth Stack / Utility), collapsible 260→64px, portalled account dropdown with plan badge + trial countdown + logout. Broadcasts <code>--evox-sidebar-w</code> . Exports internal <code>NavItem</code> and <code>MenuRow</code> sub-components.
<code>SidebarLayout.jsx</code>	—	Wrapper that renders <code>AppSidebar</code> plus a content area offset by the sidebar width. Used by ~23 pages.
<code>Navbar.jsx</code>	444	Top navigation for public/marketing pages and the plan-gate screen.
<code>ToolTopBar.jsx</code>	—	Compact page header for tool pages (title, back link, actions).
<code>StrategyTopBar.jsx</code>	—	Variant of the top bar used by the strategy section.
<code>JourneyFooter.jsx</code>	—	Renders the 12-step CMO journey progress strip. Mirrors <code>JOURNEY_STEPS</code> in <code>Lib/journeyHandoff.js</code> .

Authentication & access control

File	Lines	Purpose
<code>AuthProvider.jsx</code>	26	Creates <code>AuthContext</code> . Calls <code>useEvokeSession()</code> , maps the profile through <code>profileToUser()</code> , memoises <code>{ profile, status, user }</code> . Also renders <code>OAuthCallbackHandler</code> so callbacks are handled app-wide.
<code>OAuthCallbackHandler.jsx</code>	—	Detects OAuth return parameters in the URL and completes the connection flow. Mounted globally inside <code>AuthProvider</code> .

File	Lines	Purpose
<code>PlanGate.jsx</code>	213	Blocks a page when the user's plan is too low. Renders the real page blurred behind a lock card showing the required plan, its price, and its feature highlights, with an upgrade CTA (Stripe link if configured, otherwise the package page). Bypassed entirely on localhost (<code>IS_DEV</code>) and controllable via the <code>PLAN_GATES_ENABLED</code> flag (currently <code>true</code> , re-enabled 2026-08-04). Holds <code>PLAN_PRICES</code> (\$30 / \$100 / Custom) and <code>STRIPE_LINKS</code> .
<code>UpgradeModal.jsx</code>	—	Inline upgrade prompt for gated actions inside an otherwise-accessible page.

Onboarding

File	Lines	Purpose
<code>OnboardingWizard.jsx</code>	588	The live onboarding flow. Six steps: Cognitive System Tuning → Brand System Definition → Connect System API → Marketing Plan Tuning → Neural Network Ingestion → Cognitive Audit Report. Holds <code>INDUSTRY_OPTIONS</code> , <code>PORTFOLIO_OPTIONS</code> , <code>VOICE_OPTIONS</code> , <code>VALUE_TAGS</code> , <code>INTEGRATIONS</code> (10 toggles), <code>GOAL_OPTIONS</code> , <code>AGENT_OPTIONS</code> , and <code>PLAN_KEY_MAP</code> (maps PricingPage keys → internal plan keys). Writes to Firestore on finish.
<code>BrandSetupModal.jsx</code>	525	Modal variant of brand setup. Also exports <code>getRecommendations()</code> , which <code>OnboardingWizard</code> imports — so this file is a dependency even where the modal itself isn't shown.
<code>OnboardingModal.jsx</code>	684	Legacy chat-style onboarding modal. Superseded by <code>OnboardingWizard</code> ; kept for the retired flow.
<code>ProductTour.jsx</code>	—	First-visit guided tour. Anchors to <code>data-tour</code> attributes on sidebar items (<code>nav-connect-channels</code> , <code>nav-agents-hub</code> , <code>nav-campaigns</code> , <code>nav-campaign-execution</code>). Persists dismissal via <code>markTourSeen()</code> .

Feature components

File	Lines	Purpose
<code>BannerGenerator.jsx</code>	452	UI for AI banner generation — prompt input, provider selection, preview, download. Backed by <code>services/bannerService.js</code> .
<code>EventBannerGenerator.jsx</code>	—	Event-specific banner variant (date/time/venue/QR overlays).
<code>Chatbot.jsx</code>	480	Floating assistant rendered on every page (mounted in <code>App.jsx</code> outside <code><Routes></code>). Calls the AI generation endpoint for answers.
<code>ProductLaunchModal.jsx</code>	498	Guided product-launch campaign builder.
<code>LoadingSpinner.jsx</code>	—	Shared loading indicator.

EGT token / wallet (Evoke Gratitude Token)

File	Lines	Purpose
<code>EgtWalletHeader.jsx</code>	360	Displays the user's EGT wallet balance in the header. Reads the wallet address from the SSO cookie and the balance from an on-chain RPC endpoint.
<code>GratitudeToken.jsx</code>	—	EGT token display/award component.

6. Hooks (`src/hooks/`)

File	Exports	Purpose
<code>useAuth.js</code>	<code>useAuth()</code>	Reads <code>AuthContext</code> . Returns <code>{ profile, status, user }</code> . The hook every page should use to get the current user.
<code>useEvokeSession.js</code>	<code>useEvokeSession()</code>	The SSO session engine. Subscribes to the <code>evoke_user</code> cookie via <code>useSyncExternalStore</code> , bootstraps from the backend once, and falls back to the hard-coded <code>DEV_PROFILE</code> on localhost. Returns <code>{ profile, status }</code> where status is <code>'loading' \ 'authenticated' \ 'unauthenticated'</code> .
<code>useRequireAuth.js</code>	<code>useRequireAuth()</code>	Redirects to the accounts portal if the user isn't authenticated.

File	Exports	Purpose
<code>useUserPlan.js</code>	<code>useUserPlan()</code>	Fetches the user's plan from Firestore (<code>userPlan</code> ?? <code>selectedPlan</code> ?? <code>'free'</code>) and computes <code>trialDaysLeft</code> from <code>trialStartedAt</code> against a 14-day <code>TRIAL_DAYS</code> constant. Returns <code>{ plan, loading, trialDaysLeft }</code> .

7. Libraries (`src/lib/`)

Pure logic — no React rendering except where noted.

File	Exports	Purpose
<code>session.js</code> (397 lines)	<code>EVOKE_SESSION_EVENT</code> , <code>subscribeSession</code> , <code>decodeJwtPayload</code> , <code>getEvokeUser</code> , <code>getEvokeUserProfile</code> , <code>getEvokeUserProfileSnapshot</code> , <code>isLoggedInIn</code> , <code>buildAccountsLoginUrl</code> , <code>setLoggedInData</code> , <code>clearLoggedInData</code> , <code>establishAuthSessionFromOAuthPayload</code> , <code>fetchSessionFromBackend</code> , <code>signOut</code>	The cross-domain SSO layer. Reads/writes the <code>evoke_user</code> cookie at <code>.evokemarketplace.com</code> , decodes the JWT to compute cookie max-age, caches the parsed profile so <code>useSyncExternalStore</code> doesn't thrash, tracks a sign-out marker with a 5-minute grace period, and handles best-effort backend sign-out with a 4s timeout. Mirrors the same file in <code>converters_frontend</code> and <code>evoke_auth</code> .
<code>authUtils.js</code>	<code>profileToUser</code> , <code>redirectToLogin</code>	Maps an SSO profile to <code>{ uid, displayName, email, custID, token, firstName, lastName }</code> . <code>uid = 'sso_' + custID</code> (falls back to email).
<code>planGate.js</code> (177 lines)	<code>PLANS</code> , <code>PLAN_LABELS</code> , <code>PLAN_COLORS</code> , <code>PLAN_TAGS</code> , <code>FEATURE_PLAN</code> , <code>canAccess</code> , <code>requiredPlanFor</code> , <code>upgradeLabel</code> , <code>PLAN_HIGHLIGHTS</code> , <code>ROUTE_PLAN</code>	The plan/feature matrix. <code>PLANS</code> is ordered (<code>free</code> → <code>package-a</code> → <code>package-b</code> → <code>package-c</code>) and access is an index comparison, which makes plans cumulative. <code>FEATURE_PLAN</code> maps ~25 feature keys to plans; <code>ROUTE_PLAN</code> maps routes to plans; <code>PLAN_HIGHLIGHTS</code> drives the upgrade card copy.
<code>journeyHandoff.js</code>	<code>JOURNEY_STEPS</code> , <code>mapStrategyTypeToCampaignHub</code> , <code>strategyToCampaignHub</code> , <code>audienceToCaptionSuite</code> , <code>captionSuiteToCreativeAsset</code>	The canonical 12-step CMO journey plus mappers that shape one agent's output into the next agent's pre-fill payload. Extend <code>JOURNEY_STEP_MAPPERS</code> as more hops are wired.
<code>recommendations.jsx</code>	<code>getRecommendedActions</code>	Scores a pool of suggested next actions against the Brand Knowledge Base (industry, audience type, objectives) and returns the top N. Industry-aware — e-commerce gets product/visual suggestions, B2B gets email-drip/LinkedIn. Shared by the Dashboard and the Brand KB completion screen so recommendations are identical in both. Returns JSX icons, hence <code>.jsx</code> .
<code>campaignPrefill.js</code>	<code>buildCampaignPrefill</code>	Builds pre-filled campaign form values from prior context.
<code>swot.jsx</code>	<code>parseSwot</code> , <code>SwotGrid</code> , <code>timeAgo</code>	Parses SWOT text output into structured quadrants and renders the grid.
<code>apiAuth.js</code>	<code>authedFetch</code>	Calls the Supabase-backed Node API routes. Attaches a Firebase ID token as <code>Authorization: Bearer</code> when available; on localhost falls back to passing the plain <code>uid</code> (mirrors <code>api/_lib/verifyUser.js</code>). For <code>GET</code> , appends <code>?uid=</code> ; for others, merges <code>uid</code> into the JSON body.
<code>supabaseClient.js</code>	<code>supabase</code>	Browser Supabase client built from <code>VITE_SUPABASE_URL</code> + <code>VITE_SUPABASE_ANON_KEY</code> .
<code>gtag.js</code>	<code>gtagEvent</code> , <code>trackPurchase</code> , <code>trackSignup</code>	Google Ads conversion tracking wrappers around the global <code>gtag</code> .
<code>evokeUserCookie.js</code>	<code>readEvokeUserWalletAddressFromDocument</code>	Extracts the EGT wallet address from the SSO cookie.

File	Exports	Purpose
<code>egtRewardPoolBalance.js</code>	(balance reader)	Reads the EGT reward-pool balance from the configured RPC endpoint.
<code>formatEgtBalance.js</code>	<code>formatEgtBalanceDisplay</code>	Formats an EGT balance for display.

8. Services (`src/services/`)

The data-access layer. **Pages should not talk to Firestore or n8n directly — they go through here.**

`userService.js` — user profile, tokens, social accounts

Function	Purpose
<code>getOrCreateUser(uid, displayName, email)</code>	Fetches <code>users/{uid}</code> ; creates it with defaults on first login (zeroed <code>tokenBalance</code> , empty <code>socialAccounts</code> for facebook/instagram/linkedin/twitter/whatsapp/gmail, <code>onboardingComplete: false</code>)
<code>saveChatOnboardingData(uid, data)</code>	Saves chat-onboarding answers. Does not mark onboarding complete
<code>saveOnboardingData(uid, data)</code>	Saves answers and sets <code>onboardingComplete: true</code>
<code>saveSelectedPlan(uid, planKey)</code>	Writes both <code>selectedPlan</code> and <code>userPlan</code>
<code>markTourSeen(uid)</code>	Sets <code>productTourSeen: true</code>
<code>getUserData(uid)</code>	Returns the full user document or <code>null</code>
<code>getTokenBalance(uid)</code>	Returns <code>tokenBalance</code> (default 0)
<code>addTokens(uid, amount)</code>	Firestore <code>increment(amount)</code> — used after purchase
<code>deductToken(uid)</code>	Throws <code>'Insufficient tokens'</code> if balance < 1, else <code>increment(-1)</code>
<code>saveSocialAccount(uid, platform, accountData)</code>	Writes <code>socialAccounts.<platform></code> with <code>connected: true</code>
<code>disconnectSocialAccount(uid, platform)</code>	Sets <code>socialAccounts.<platform> = { connected: false }</code>
<code>getSocialAccounts(uid)</code>	Returns the whole <code>socialAccounts</code> map
<code>TOKEN_PACKAGES</code> (const)	Three purchasable packs: Starter 10/\$9.99, Growth 20/\$17.99, Pro 35/\$24.99

`contentService.js` — the `content_items` collection

Function	Purpose
<code>saveContentItems(userId, campaignMeta, items, status)</code>	Batch-writes generated items in one Firestore commit. Returns <code>{ [key]: docId }</code> so callers can update them later
<code>scheduleCampaignDays(userId, campaignId, campaignMeta, dayPayloads)</code>	Writes one row per campaign day 2..N with <code>status: 'scheduled'</code> , <code>requiresApproval: false</code> , and a real <code>scheduledAt</code> . This is what the n8n cron queries. Replaced the old browser- <code>setTimeout</code> approach that died when the tab closed
<code>updateContentItem(id, data)</code>	Patches any fields + stamps <code>updatedAt</code>
<code>markItemsPublished(ids, finalTexts)</code>	Batch-marks published; <code>finalTexts</code> persists user edits made between generation and launch
<code>setItemStatus(id, status, extra)</code>	Status transition; auto-sets <code>publishedAt</code> when status is <code>'published'</code>
<code>scheduleItem(id, scheduledAt)</code>	Sets <code>status: 'scheduled'</code> and <code>requiresApproval: false</code> (a human already approved it in <code>/queue</code>)
<code>getContentItems(userId, status?)</code>	Fetches the library, sorted newest-first client-side to avoid needing a composite Firestore index

ghlService.js — GoHighLevel social publishing (the live social path)

Export	Purpose
GHL_PLATFORMS	['facebook', 'instagram', 'linkedin', 'tiktok', 'threads', 'google']
getGhlAccount(uid)	Returns the user's GHL location, or <code>null</code>
ensureGhlLocation(uid, profile)	Tries to provision a dedicated sub-account via n8n; on failure falls back to <code>SHARED_LOCATION_ID</code> . Safe to call on every login
startSocialConnect(uid, platform)	Gets an OAuth start URL from n8n, opens a popup, resolves on <code>postMessage</code> from a trusted origin allow-list , then lists authorised pages
attachSocialPage(uid, platform, accountId, page)	Records the chosen page under <code>socialAccounts.<platform></code> with <code>viaGhl: true</code> and <code>ghlAccountId: page.originId</code>
disconnectSocial(uid, platform)	Clears the connection
publishSocialPost(uid, { platforms, caption, mediaUrls, scheduleDate })	Resolves each platform's <code>ghlAccountId</code> , then POSTs to n8n <code>/webhook/evoke-social-post</code> . Omit <code>scheduleDate</code> to publish immediately

Two safety behaviours worth preserving: - **Trusted popup origins** — `postMessage` is only accepted from `services.leadconnectorhq.com` and `app.gohighlevel.com`. - **scoped flag** — `listConnectedPages()` returns `scoped: true` only when the page list could be narrowed to the account that just signed in. When `false`, the UI must **not** auto-select; on the shared workspace that would silently link another user's page.

metaAdsService.js / googleAdsService.js — paid ads

Service	Functions
metaAdsService.js	<code>getMetaAdsAccount</code> , <code>connectMetaAdsCallback</code> , <code>disconnectMetaAds</code> , <code>createMetaAdsBoost</code> — plus <code>META_ADS_OAUTH_WEBHOOK</code> , <code>META_ADS_BOOST_WEBHOOK</code>
googleAdsService.js	<code>getGoogleAdsAccount</code> , <code>createGoogleAdsCampaign</code> , <code>getGoogleAdsMetrics</code> , <code>extractAdsContent</code> — plus <code>GOOGLE_ADS_CREATE_WEBHOOK</code> , <code>GOOGLE_ADS_METRICS_WEBHOOK</code>

Remaining services

File	Exports	Purpose
knowledgeBaseService.js	<code>getKnowledgeBase</code> , <code>saveKnowledgeBase</code> , <code>appendJourneyOutput</code>	Brand Knowledge Base persistence. Stores it as a nested knowledgeBase field on users/{uid} , not a subcollection — deliberately, to avoid subcollection permission issues. <code>appendJourneyOutput(uid, step, summary)</code> records each journey step's output keyed by step name. Note the header comment: a Supabase-backed version was built and then reverted on 2026-08-04
bannerService.js (353 lines)	<code>generateEventPosterWithCanvas</code> , <code>generateBanner</code> , <code>downloadBanner</code> , <code>sanitizeBannerFilename</code>	Banner generation — either composed locally on an HTML <code><canvas></code> or generated by calling <code>/api/generate-banner</code>
crmService.js	<code>calcScore</code> , <code>getContacts</code> , <code>addContact</code> , <code>updateContact</code> , <code>deleteContact</code>	CRM contacts at <code>users/{uid}/contacts/{contactId}</code> . <code>calcScore</code> computes a lead score
teamService.js	<code>getMembers</code> , <code>ensureOwnerMember</code> , <code>addMember</code> , <code>updateMemberRole</code> , <code>removeMember</code> , <code>getInvites</code> , <code>createInvite</code> , <code>resendInvite</code> , <code>cancelInvite</code>	Team management at <code>users/{uid}/teamMembers/{memberId}</code> . Roles: <code>owner \ admin \ editor \ viewer</code>
governanceService.js	<code>saveAuditEntry</code> , <code>getAuditLog</code>	Firestore-backed audit trail for the Brand Governance Agent, so decisions survive a reload instead of living only in React state
socialInboxService.js	<code>saveThreadReply</code> , <code>getSavedThreads</code>	Persists replies only . There is no live social message source wired in — <code>SocialInbox.jsx</code> shows example threads as the base data, and this service layers the user's real replies on top

File	Exports	Purpose
<code>eventService.js</code>	<code>buildEventSlug</code> , <code>saveEventPage</code> , <code>getEventPage</code> , <code>generateEventHtml</code> , <code>downloadEventHtml</code>	Event landing-page generation and hosting
<code>emailService.js</code>	<code>sendEmail</code>	Thin client for <code>/api/send-email</code> (Resend)
<code>qrCodeService.js</code>	<code>generateQRCode</code> , <code>addQRCodeToCanvas</code>	QR codes via the free QR Server API — no key required. Used to overlay registration links on event banners

9. Pages (`src/pages/`)

75 files. Grouped by function; line counts shown for the substantial ones.

Public & marketing

File	Lines	Purpose
<code>Landing.jsx</code>	4,392	Public marketing home page. Includes a large inline dashboard mockup section built to a Figma spec
<code>PricingPage.jsx</code>	1,588	Four-tier pricing page. Its plan keys map to internal keys via <code>PLAN_KEY_MAP</code> in <code>OnboardingWizard.jsx</code>
<code>PlansPage.jsx</code>	392	Plan comparison/selection
<code>FreePlanPage.jsx</code> , <code>PackageAPage.jsx</code> , <code>PackageBPage.jsx</code> (801), <code>PackageCPage.jsx</code> (665)	—	Per-tier detail pages. <code>PackageBPage</code> documents its own scope: Content Production + Video Creation + 30-Day Planning
<code>Privacy.jsx</code> , <code>Terms.jsx</code>	—	Legal pages
<code>SignIn.jsx</code>	—	Legacy route; redirects to the Evoke accounts portal for SSO

Onboarding & brand

File	Lines	Purpose
<code>BrandProfilePage.jsx</code>	—	Hosts <code>OnboardingWizard</code> . Also serves <code>?tab=settings</code> from the sidebar's Settings link
<code>BrandKnowledgeBase.jsx</code>	585	Brand KB viewer/editor (Package A)
<code>Onboarding.jsx</code>	401	Legacy chat onboarding (route retired)
<code>SetupPage.jsx</code>	473	Legacy setup page with a recommendation engine (route retired)

Dashboard & strategy

File	Lines	Purpose
<code>DashboardPage.jsx</code>	761	Main dashboard. Renders scored recommendations from <code>getRecommendedActions()</code>
<code>StrategyHome.jsx</code>	—	Strategy section landing page
<code>MarketingStrategyPage.jsx</code>	591	Annual / quarterly / monthly marketing plans
<code>CompetitorIntelPage.jsx</code>	—	Competitor intelligence
<code>SwotAnalysisPage.jsx</code>	—	SWOT analysis, rendered via <code>lib/swot.jsx</code>
<code>MarketingHealthPage.jsx</code>	466	Marketing health score
<code>KpiRecommendationsPage.jsx</code>	639	KPI recommendations

Campaigns

File	Lines	Purpose
<code>CampaignForm.jsx</code>	4,488	The largest and most important page. Multi-type campaign builder — collects inputs, posts to the n8n <code>evolve-cmo</code> webhook, saves generated items to <code>content_items</code> , and schedules days 2..N
<code>Results.jsx</code>	2,116	Displays generated campaign output; allows editing before publish
<code>CampaignsPage.jsx</code>	—	Campaign list
<code>NewCampaignWizardPage.jsx</code>	431	Guided new-campaign wizard
<code>CampaignHub.jsx</code>	—	Campaign planning hub (journey step 3)
<code>CampaignPerformancePage.jsx</code>	—	Per-campaign performance

Content generation

File	Lines
<code>CaptionSuitePage.jsx</code>	444
<code>ReelScriptsPage.jsx</code>	604
<code>ContentGenerationPage.jsx</code>	694
<code>CopywritingAgentPage.jsx</code>	619
<code>ProductDescription.jsx</code>	393
<code>BlogGeneratorPage.jsx</code>	—
<code>EmailComposerPage.jsx</code>	—
<code>ContentStudioHubPage.jsx</code>	—

Creative & media

File	Lines	Purpose
<code>CreativeAssetPage.jsx</code>	808	Creative Asset Generator
<code>ImageToolPage.jsx</code>	684	Serves 5 routes — angles, 360°, SEO, lifestyle, 3D
<code>AIImageGeneratorPage.jsx</code>	—	AI image generation
<code>VideoGenerationPage.jsx</code>	641	Video generation
<code>VideoStudioHubPage.jsx</code>	—	Video studio hub
<code>CreativeStudioHubPage.jsx</code>	—	Creative studio hub
<code>ProductsPage.jsx</code>	540	Product/asset library

Agents

File	Lines	Purpose
<code>AgentsHub.jsx</code>	375	AI agent catalogue
<code>CmoAgentOverviewPage.jsx</code>	522	Per-agent detail — serves <code>/agent/:type</code>
<code>SeoAgentPage.jsx</code>	634	SEO Agent
<code>SeoIntelligenceCenterPage.jsx</code>	—	SEO Intelligence Center. Its header comment states plainly that no SEMrush/Ahrefs/Search Console integration exists, so its metrics have no real data source
<code>EmailMarketingPage.jsx</code>	542	Email Marketing Agent

File	Lines	Purpose
<code>CSuitePage.jsx</code>	472	Serves all four C-suite routes (<code>/ai-cfo</code> , <code>/ai-cto</code> , <code>/ai-ceo</code> , <code>/ai-cro</code>)
<code>CompliancePage.jsx</code>	—	Compliance Agent
<code>BrandGovernancePage.jsx</code>	431	Brand Governance with a persisted audit trail

Social & publishing

File	Lines	Purpose
<code>ConnectAccounts.jsx</code>	2,273	Account connection UI for every platform
<code>PostContent.jsx</code>	1,307	Compose and publish immediately
<code>ApprovalQueue.jsx</code>	635	Draft review → approve → schedule
<code>SocialInbox.jsx</code>	401	Unified inbox. Explicitly labelled in-UI as example conversations — no live inbox source is connected; replies are saved and "Suggested Reply" is a real AI call
<code>SocialMediaManagerPage.jsx</code>	—	Social media manager hub
<code>SocialCalendarPage.jsx</code>	—	Content calendar
<code>EventbritePost.jsx</code>	598	Eventbrite event creation & publishing

Ads

File	Lines
<code>MetaAdsBoost.jsx</code>	707
<code>AdsCenterHubPage.jsx</code>	—
<code>MarketingExecutionPage.jsx</code>	610

Analytics & reporting

File	Lines
<code>AnalyticsDashboard.jsx</code>	728
<code>ExecutiveReportPage.jsx</code>	548
<code>MarketingAttributionPage.jsx</code>	576
<code>ABTestingPage.jsx</code>	513
<code>TrendAnalysis.jsx</code>	434
<code>AudienceBuilder.jsx</code>	601

Business operations

File	Lines	Purpose
<code>CrmPage.jsx</code>	693	CRM & lifecycle
<code>TeamManagement.jsx</code>	440	Team members, roles, invites
<code>PartnerSharing.jsx</code>	308	Partner sharing
<code>Tokens.jsx</code>	872	Token balance & packages
<code>Purchase.jsx</code>	—	Checkout. Loads the Razorpay script dynamically

Dev-only

File	Purpose
DevResetPage.jsx	Resets local dev state. Localhost only
SupabaseTestPage.jsx	Inserts a row and reads it back to verify the Supabase connection. Explicitly not part of the real data layer. Localhost only

10. Configuration modules

src/config.js

Resolves the three main n8n webhook URLs with a three-level fallback:

```
export const WEBHOOK_URL = import.meta.env.VITE_N8N_WEBHOOK
|| (import.meta.env.DEV ? '/n8n-webhook/evoke-cmo' : `${N8N_BASE}/webhook/evoke-cmo`)
```

In dev the /n8n-webhook prefix hits the Vite proxy (avoiding CORS); in production it hits VITE_N8N_BASE directly.

Export	Default path
WEBHOOK_URL	/webhook/evoke-cmo — main campaign
DAY_WEBHOOK_URL	/webhook/evoke-cmo-day — single-day content
AGENT_WEBHOOK_URL	/webhook/evoke-agents — AI agents
GOOGLE_ADS_OAUTH_WEBHOOK	/webhook/google-ads-oauth
GOOGLE_ADS_CREATE_WEBHOOK	/webhook/google-ads-create-campaign
GOOGLE_ADS_METRICS_WEBHOOK	/webhook/google-ads-metrics
ADMIN_EMAIL	Approval-email recipient
META_API_BASE	Meta / Evoke Marketing FastAPI backend base URL

The file opens with a comment block explaining **how to switch n8n accounts** — create a new n8n trial, import the workflow JSON, paste the new webhook URLs (or set the VITE_ vars), and toggle the workflow active.

src/firebase.js

Initialises Firebase from VITE_FIREBASE_* env vars (never hardcoded) and exports: auth, db (Firestore), storage, plus googleProvider, facebookProvider, twitterProvider and a re-exported signInWithPopup. Uses the getApp().length === 0 ? initializeApp(...) : getApp() guard so hot reload doesn't double-initialise.

Part 3 — Backend Reference

The backend has **three distinct layers**. Knowing which one you're in matters, because they run in different places and behave differently.

Layer	Where it runs	When it's active
1. Vercel Functions (<code>api/</code>)	Vercel Edge / Node runtime	Production, and under <code>vercel dev</code>
2. Vite dev middleware (<code>vite.config.js</code>)	Your local machine	Local development only (<code>npm run dev</code>)
3. n8n workflow	<code>n8n-zvxi.srv1837606.hstgr.cloud</code>	Always — same instance for dev and production

⚠ **The most important gotcha in this codebase:** plain `vite` does **not** run the `api/` directory. Those are Vercel functions. Locally, only the routes reimplemented in `vite.config.js` exist. Any `api/` route without a Vite equivalent returns **404 locally** — which is exactly how the Supabase Brand-KB migration silently failed and was reverted on 2026-08-04. To exercise the real functions locally you must run `vercel dev`.

Layer 1 — Vercel Functions (`api/`)

Seven routes. Six run on the **Edge runtime**; one needs Node.

File	Runtime	Method	Auth	Purpose
<code>generate.js</code>	Edge	POST	none	Groq chat-completions proxy
<code>generate-campaign.js</code>	Edge	POST	none	Full campaign copy package
<code>generate-banner.js</code>	Edge	POST	none	Image generation (3 providers)
<code>gemini-image.js</code>	Edge	POST	none	Gemini image generation
<code>eventbrite.js</code>	Edge	POST	caller-supplied token	Eventbrite API proxy
<code>send-email.js</code>	Edge	POST	none	Transactional email via Resend
<code>brand-kb.js</code>	Node	GET/POST	Firebase ID token	Brand KB read/write via Supabase

All seven handle `OPTIONS` preflight and set `Access-Control-Allow-Origin: *`.

`api/generate.js` — Groq proxy

```
POST /api/generate
Body: <any Groq chat-completions payload, passed through verbatim>
→ 200 <Groq response, passed through verbatim>
```

Exists purely to keep the Groq API key server-side and avoid browser CORS. Reads `process.env.VITE_GROQ_API_KEY`; returns 500 if unset.

This is the endpoint most content-generation pages call. The caller controls the model and the prompt — the proxy adds no logic of its own.

`api/generate-campaign.js` — full campaign package

The headless/backend-to-backend entry point. Unlike the browser flow (which fires the n8n webhook and gets an acknowledgement), this returns the **actual generated copy** in the response. Built for integrations like the marketplace calling the API directly.

```
POST /api/generate-campaign
Body: {
  campaignType?, name*, description*, goal?, targetAudience?,
  brandName?, date?, time?, location?, eventUrl?, campaignDays?
}
→ 200 { success: true, data: { ...15 fields... } }
→ 400 { success: false, error } — name and description are required
→ 502 { success: false, error } — network error / unparseable AI output
```

Model: Groq `llama-3.1-8b-instant`, temperature 0.7, max_tokens 3500.

Output schema — the prompt demands exactly these 15 fields:

```
campaignName · emailSubject · emailBody · linkedinPost · instagramCaption
facebookPost · whatsappMessage · smsMessage · seoTitle · seoDescription
adHeadline · adBody · tiktokCaption · campaignCalendar · positioningStatement
```

Two behaviours worth preserving:

1. **extractJson()** — a four-strategy parser. LLMs wrap JSON unpredictably, so it tries, in order: ````json` fence → any ````` fence → first block → from the first { to end-of-string with a } appended. If all four still fail to `JSON.parse`, it escapes raw newlines/tabs *inside string literals* and retries. This matches the frontend's parser exactly so behaviour is identical on both paths.
2. **Groq 401 is remapped to 500.** A bad server-side API key must never surface to the client as "unauthorized" — that would read as a user auth problem.

```
const status = groqRes.status === 401 ? 500 : groqRes.status
```

Event campaigns get special handling: when `campaignType` is `event` / `event_full` and an `eventUrl` is present, the prompt instructs the model to embed the registration URL in every social post, the email body and the WhatsApp message.

`api/generate-banner.js` — image generation, three providers

```
POST /api/generate-banner
Body: { prompt*, provider?: 'gemini'|'dalle'|'pollinations', width?, height? }
→ 200 { base64Image, mimeType, provider, revised_prompt? }
```

provider	Model	Key required	Notes
gemini (default)	gemini-2.0-flash-exp	GEMINI_API_KEY	responseModalities: ['IMAGE', 'TEXT']; image comes back in <code>inlineData</code>
dalle	gpt-image-1 at 1536×1024, quality high	OPENAI_API_KEY	Returns base64 directly via <code>b64_json</code>
pollinations	flux	none	Free fallback

Always returns **base64**, never a URL — so the caller never has to deal with a short-lived provider CDN link.

Naming caveat: the function is called `generateWithDalle` and the provider string is `dalle`, but the model actually requested is `gpt-image-1`. The dev-mode equivalent in `vite.config.js` still requests `dall-e-3` at 1024×1024 and then *fetches the returned URL* to convert it to base64. **The two paths differ.** See [Part 8](#).

One fixed bug worth not re-introducing: the seed is `Math.floor(Math.random() * 2147483647)`. It used to be `Date.now()`, which overflows a 32-bit int and Pollinations rejects.

`api/gemini-image.js` — Gemini image generation

A narrower version of the above: Gemini only, no provider switch, no dimensions.

```
POST /api/gemini-image
Body: { prompt* }
→ 200 { base64Image, mimeType }
```

Requires `GEMINI_API_KEY`. Functionally overlapped by `generate-banner.js?provider=gemini`.

`api/eventbrite.js` — Eventbrite proxy

An action-dispatch proxy. The **caller supplies the access token**; the function holds no Eventbrite credentials of its own.

```
POST /api/eventbrite
Body: { action*, accessToken*, ...action-specific fields }
```

action	Eventbrite endpoint called
<code>create_venue</code>	<code>POST /organizations/{organizationId}/venues/</code>
<code>create_event</code>	<code>POST /organizations/{organizationId}/events/</code>
<code>create_ticket</code>	<code>POST /events/{eventId}/ticket_classes/</code>
<code>publish_event</code>	<code>POST /events/{eventId}/publish/</code>

Returns Eventbrite's response body and status verbatim. Unknown actions → 400.

`api/send-email.js` — Resend

```
POST /api/send-email
Body: { to*, subject*, html|text*, from? }
→ 200 { id, sent: true }
→ 400 - missing required fields
→ 501 - RESEND_API_KEY not configured
```

`to` accepts a string or an array. Default sender falls back through `from` → `RESEND_FROM_EMAIL` → `'EVOX AI CMO <onboarding@resend.dev>'`.

Returns **501 Not Implemented** (not 500) when the key is missing, with a message pointing at resend.com — an honest "not configured yet" rather than a generic failure.

`api/brand-kb.js` — Brand Knowledge Base (Supabase, Node runtime)

The only authenticated API route, and the only one that isn't Edge — it needs `firebase-admin`, which doesn't run on the Edge runtime.

```
GET /api/brand-kb → { data: <kb object> | null }
POST /api/brand-kb Body: { data* } → { ok: true }
→ 401 Unauthorized - no verifiable uid
```

Reads/writes the `brand_knowledge_base` Supabase table scoped to the verified uid, using `upsert(..., { onConflict: 'user_id' })`.

Current status: NOT in the live path. `src/services/knowledgeBaseService.js` documents the reversal (2026-08-04): Vercel API routes don't run under plain `vite`, and the required credentials (Firebase admin key, Supabase service-role key) were never added — so every save silently 404'd. The frontend went back to writing the KB as a nested field on the Firestore `users/{uid}` document. **Reinstate only once both credential sets exist and it has been tested under `vercel dev` or a real deployment.**

api/_lib/ — shared server-only helpers

api/_lib/supabaseAdmin.js

```
export function getSupabaseAdmin() {
  return createClient(process.env.VITE_SUPABASE_URL,
    process.env.SUPABASE_SERVICE_ROLE_KEY)
}
```

Uses the **service-role key, which bypasses Row Level Security**. The file header states the rule: *never import this from* `src/` — it must only ever run in Node API routes.

api/_lib/verifyUser.js

```
export async function verifyRequestUser(req) → uid | null
```

Environment	Behaviour
<code>NODE_ENV === 'production'</code>	Requires <code>Authorization: Bearer <Firebase ID token></code> , verifies it with <code>firebase-admin</code> , returns <code>decoded.uid</code> . Returns <code>null</code> on any failure
Otherwise	Trusts <code>req.body.uid</code> / <code>req.query.uid</code> with no verification

The file's header comment explains the design: the app's uid (`'sso_' + custID`, from `src/lib/authUtils.js`) is the *same* uid Firebase Auth signs the user in as after the SSO redirect. So a verified Firebase ID token yields a trustworthy version of the exact uid the rest of the app already uses — no second identity system needed. On localhost the SSO flow never runs (`DEV_PROFILE` has `token: null`), so the dev branch mirrors the app's existing `IS_LOCAL` / `IS_DEV` convention rather than inventing a new one.

Requires `FIREBASE_ADMIN_PROJECT_ID`, `FIREBASE_ADMIN_CLIENT_EMAIL`, `FIREBASE_ADMIN_PRIVATE_KEY` (with `\n` unescaped at runtime).

Layer 2 — Vite dev middleware (`vite.config.js`)

553 lines. The `local-api-handlers` plugin reimplements API routes as Connect middleware so `npm run dev` works without Vercel.

Routes implemented locally

Route	Mirrors	Differences from production
<code>/api/generate</code>	<code>api/generate.js</code>	None — same passthrough
<code>/api/generate-campaign</code>	<code>api/generate-campaign.js</code>	None — prompt and parser duplicated verbatim
<code>/api/generate-banner</code>	<code>api/generate-banner.js</code>	<code>dalle</code> uses <code>dall-e-3 @1024×1024 + URL fetch</code> , production uses <code>gpt-image-1 @1536×1024 + direct base64</code>
<code>/api/eventbrite</code>	<code>api/eventbrite.js</code>	None
<code>/api/run-agent</code>	(no production equivalent)	Dev-only — see below
<code>/auth/facebook/callback</code>	(no production equivalent)	Dev-only OAuth landing page; redirects to <code>/connected-accounts#facebook_code=...</code>

`/api/run-agent` — the specialist agent prompts (dev only)

This route exists **only** in `vite.config.js`. In production the equivalent work is done by the n8n `evoke-agents` webhook using Gemini.

Model: Groq `llama-3.3-70b-versatile`, temperature 0.7, max_tokens 4096.

Nine agent prompt templates in the `PROMPTS` object, each demanding a `{ title, sections: [{ heading, content }] }` JSON response:

agentType	Inputs	Produces
reddit	subreddit, topic, brandContext	3 reply drafts + subreddit targets + engagement strategy
seo	websiteUrl, targetKeyword, industry	15 keywords + 800-word blog + meta tags + snippet + linking
writer	topic, brandName, audience, tone	1200-word article + 5 social quotes + newsletter + takeaways
twitter	topic, brandName, tone	5 tweets + 7-tweet thread + 3 hooks + hashtags
linkedin_agent	topic, brandName, audience	3 post drafts (story/insight/engagement) + schedule
hackernews	topic, brandName, value	Show HN draft + 3 comments + thread targeting
geo	brandName, keywords, description	AI-citation blocks + entity strategy + 10 FAQs + schema + distribution
coding	websiteUrl, issue, techStack	Technical SEO diagnosis + 2 code fixes + perf + test checklist
ugc_videos	product, style, audience, platforms	Brief + 60s script + 30s script + 5 hooks + platform adaptations

Unknown `agentType` falls back to `seo` . If the response isn't parseable JSON, it's wrapped as a single-section result rather than failing.

Dev-server CORS proxies

Path prefix	Target	Rewrite
/n8n-webhook	https://n8n-zvxi.srv1837606.hstgr.cloud	→ /webhook
/groq-api	https://api.groq.com	→ / (legacy, kept for back-compat)
/evoke-api	https://apieksv1.evokemarketplace.com	→ /api
/pollinations	https://image.pollinations.ai	→ /

`server.port` is set to `5173` here, but `package.json`'s `dev` script passes `--port 3007` , which wins. **The app runs on 3007.**

Env loading

```
const env = loadEnv(mode, process.cwd(), '')
```

The empty prefix means **all** env vars load, not just `VITE_`-prefixed ones — so the dev middleware can read `GEMINI_API_KEY` and `OPENAI_API_KEY` .

Layer 3 — n8n

Covered in full in [Part 4](#).

Deployment configuration — `vercel.json`

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

The catch-all rewrite is what makes client-side routing work — every path serves `index.html` and React Router takes over.

Vercel resolves `/api/*` against the `api/` directory **before** applying rewrites, so the functions are still reachable despite the wildcard.

Request-flow summary

Where a given call actually lands, dev vs production:

Frontend call	Local (<code>npm run dev</code>)	Production (Vercel)
<code>/api/generate</code>	Vite middleware → Groq	Edge function → Groq
<code>/api/generate-campaign</code>	Vite middleware → Groq	Edge function → Groq
<code>/api/generate-banner</code>	Vite middleware → Gemini/OpenAI/Pollinations	Edge function → same
<code>/api/run-agent</code>	Vite middleware → Groq	404 — does not exist
<code>/api/eventbrite</code>	Vite middleware → Eventbrite	Edge function → Eventbrite
<code>/api/send-email</code>	404 — not implemented locally	Edge function → Resend
<code>/api/gemini-image</code>	404 — not implemented locally	Edge function → Gemini
<code>/api/brand-kb</code>	404 — not implemented locally	Node function → Supabase
<code>/n8n-webhook/*</code>	Vite proxy → n8n	Direct to <code>VITE_N8N_BASE</code>

Part 4 — n8n Automation Backend

This is where the real backend work happens. OAuth token exchange, social publishing, paid-ads creation and scheduled posting all live in n8n — not in this repository.

Property	Value
Workflow name	EVOKE Cmo
Export file	EVOKE-CMO-v21-GHL.json (262 KB)
Host	https://n8n-zvxi.srv1837606.hstgr.cloud
Base webhook path	/webhook/<path>
Total nodes	230
Webhook triggers	25
Schedule triggers	1 (every 15 minutes)
Status	Active

Node composition

Node type	Count	Role
httpRequest	75	Calls to every external API
code	65	Payload shaping, token handling, response normalising
respondToWebhook	33	Returns responses to the frontend
if	27	Platform/branch routing
webhook	25	Entry points
wait	2	LinkedIn video processing delays
set	1	Credential normalisation
scheduleTrigger	1	The 15-minute publisher
merge	1	Combines multi-platform publish results

1. Complete webhook index

#	Path	Method	Purpose	Called from
1	facebook-oauth	POST	Facebook OAuth token exchange	ConnectAccounts.jsx
2	instagram-oauth	POST	Instagram OAuth token exchange	ConnectAccounts.jsx
3	linkedin-oauth	POST	LinkedIn OAuth token exchange	ConnectAccounts.jsx
4	gmail-oauth	POST	Gmail OAuth token exchange	ConnectAccounts.jsx
5	eventbrite-oauth	POST	Eventbrite OAuth token exchange	ConnectAccounts.jsx
6	tiktok-oauth	POST	TikTok OAuth token exchange	ConnectAccounts.jsx
7	evoke-cmo	POST	Main campaign generate + publish	CampaignForm.jsx
8	evoke-cmo-day	POST	Publish a single campaign day	contentService / cron
9	evoke-agents	POST	AI specialist agents	Agent pages

#	Path	Method	Purpose	Called from
10	<code>evoke-cmo-product</code>	POST	Product description generation	<code>ProductDescription.jsx</code>
11	<code>google-ads-oauth</code>	POST	Google Ads OAuth	<code>googleAdsService.js</code>
12	<code>google-ads-create-campaign</code>	POST	Create a Google Ads campaign	<code>googleAdsService.js</code>
13	<code>google-ads-metrics</code>	POST	Google Ads metrics	<code>googleAdsService.js</code>
14	<code>meta-ads-oauth</code>	POST	Meta Ads OAuth	<code>metaAdsService.js</code>
15	<code>meta-ads-boost</code>	POST	Create a Meta ad campaign	<code>metaAdsService.js</code>
16	<code>twilio-status-callback</code>	POST	Twilio delivery status	Twilio (inbound)
17	<code>meta-whatsapp-webhook</code>	GET	WhatsApp webhook verification	Meta (inbound)
18	<code>meta-whatsapp-webhook</code>	POST	WhatsApp inbound events	Meta (inbound)
19	<code>evoke-ghl-token</code>	POST	Get/refresh the GHL agency token	Internal (n8n → n8n)
20	<code>evoke-oauth-callback</code>	GET	GHL Marketplace app install callback	GoHighLevel (inbound)
21	<code>evoke-location-create</code>	POST	Create a GHL sub-account	<code>ghlService.js</code>
22	<code>evoke-social-start</code>	POST	Begin social OAuth via GHL	<code>ghlService.js</code>
23	<code>evoke-social-accounts</code>	POST	List / attach GHL social pages	<code>ghlService.js</code>
24	<code>evoke-social-post</code>	POST	Publish to social accounts	<code>ghlService.js</code>
25	<code>evoke-set-pit</code>	POST	Store the GHL Private Integration Token	Manual (admin)

2. Social OAuth flows (6 webhooks)

All six follow the same four-stage shape:

```
Webhook → Extract Params (code) → Exchange Code (HTTP)
          → [Get Long-lived Token] → Get Profile (HTTP)
          → Build Response (code) → Respond to Webhook
```

The browser never sees a client secret — the code-for-token exchange happens entirely inside n8n.

Per-platform detail

Platform	Token exchange endpoint	Profile / entity fetched	Extra steps
Facebook	<code>graph.facebook.com/v21.0/oauth/access_token</code>	<code>/me/accounts</code> (Pages)	Short → long-lived token exchange, then <code>FB Get Instagram</code> fetches the linked IG business account from the page
Instagram	<code>api.instagram.com/oauth/access_token</code>	<code>graph.instagram.com/v21.0/me</code>	Short → long-lived via <code>graph.instagram.com/access_token</code>
LinkedIn	<code>linkedin.com/oauth/v2/accessToken</code>	<code>api.linkedin.com/v2/userinfo</code>	Has a dedicated <code>LI Error Response</code> branch
Gmail	<code>oauth2.googleapis.com/token</code>	<code>googleapis.com/oauth2/v2/userinfo</code>	Stores the refresh token for later sends
Eventbrite	<code>eventbrite.com/oauth/token</code>	<code>/v3/users/me/organizations/</code>	Has a dedicated error branch
TikTok	<code>open.tiktokapis.com/v2/oauth/token/</code>	<code>/v2/user/info/?fields=display_name,open_id</code>	Has a code-node error handler

The response is written back to Firestore by the frontend via `saveSocialAccount(uid, platform, accountData)`.

3. The main campaign engine — evoke-cmo

The single largest flow in the workflow — roughly 60 nodes. Called by `CampaignForm.jsx`.

Flow shape

```
Form Webhook - AI CMO
├─ Normalize & Extract Credentials1 (set)
│   └─ Flatten User Credentials1 (code)
│       └─ Respond to Webhook ← responds IMMEDIATELY, before generation
│           └─ Build Smart Content Prompt1
│               └─ Needs Content Gen? (if)
│                   └─ Gemini Generate Content1 → gemini-2.0-flash
│                       └─ Extract & Merge Content1
│                           └─ [image generation] → [per-platform publishing fan-out]
```

Fire-and-forget by design. `Respond to Webhook` runs immediately after credential flattening, so the browser gets an acknowledgement in ~1s while generation and publishing continue server-side for a minute or more. The UI must not wait for publish results — it reads them from Firestore afterwards.

Image generation sub-flow

```
Build Gemini Prompt1
├─ Skip If Has Image1 (if) ← don't regenerate if the user supplied one
│   └─ Call Gemini API1 → gemini-2.0-flash-exp
│       └─ Extract Image Data1
│           └─ Upload to ImgBB1 → api.imgbb.com/1/upload
│               └─ Set Image URL1
```

ImgBB is used because Meta and LinkedIn need a **publicly reachable URL**, not base64.

Publishing fan-out

Each platform is a conditional branch off the generated content:

Platform	Branch	API path
LinkedIn (text)	<code>Has LinkedIn?1</code> → <code>LI Post Text Only1</code>	<code>POST /v2/ugcPosts</code>
LinkedIn (image)	<code>Has LI Image?1</code> → register → download → upload → post	<code>/v2/assets?action=registerUpload</code> then <code>/v2/ugcPosts</code>
LinkedIn (video)	<code>LI Check Is Video</code> → 7-node sub-flow	<code>/rest/videos?action=initializeUpload</code> → upload bytes → <code>finalizeUpload</code> → <code>Wait</code> → <code>POST /rest/posts</code>
Instagram	<code>Has Instagram?1</code>	<code>/igBusinessId/media</code> then <code>/media_publish</code> (two-step, required by Meta)
Facebook (image)	<code>Has FB Image?1</code>	<code>/pageId/photos</code>
Facebook (video)	<code>Has FB Video?1</code>	<code>/pageId/videos</code>
Facebook (text)	fallback	<code>/pageId/feed</code>
WhatsApp	<code>Has WhatsApp?1</code> → split recipients	Twilio <code>/Messages.json</code>
SMS	<code>Has SMS?1</code> → split recipients	Twilio <code>/Messages.json</code>
Email	<code>Has Email?1</code>	Refresh Gmail token → build MIME → <code>gmail.googleapis.com/.../messages/send</code>
Eventbrite	<code>Has Eventbrite?1</code>	<code>/v3/organizations/{orgId}/events/</code>
Luma	<code>Has Luma?1</code>	<code>api.lu.ma/v1/calendar/create-event</code>

Platform	Branch	API path
Meetup	Has Meetup?1	(branch present)

LinkedIn video is the most complex path in the entire workflow — six HTTP calls plus two `Wait` nodes for asynchronous processing on LinkedIn's side.

WhatsApp and SMS both pass through a "Split Recipients" code node so one campaign fans out to a list of numbers.

4. The AI agents endpoint — `evoke-agents`

```

Agents Webhook
├── Build Agent Prompt (code - selects from 9 templates by agentType)
│   └── Call Gemini Agent → gemini-2.0-flash
│       ├── Format Agent Response → Respond to Agent Request
│       └── Respond Error2

```

`Build Agent Prompt` holds the same nine agent templates as the dev-only `/api/run-agent` route (`reddit`, `seo`, `writer`, `twitter`, `linkedin_agent`, `hackernews`, `geo`, `coding`, `ugc_videos`), defaulting to `seo`. Each demands `{ title, sections: [{ heading, content }] }`.

Note the provider split: production agents run on **Gemini** via n8n; local dev agents run on **Groq** `llama-3.3-70b-versatile` via Vite middleware. Same prompts, different models — outputs will differ between environments.

Also note: the `Call Gemini Agent` node has its API key embedded directly in the URL query string rather than in a credential. See [Part 8](#).

5. GoHighLevel integration (7 webhooks)

GoHighLevel (GHL) is the execution engine for social publishing. It holds the Meta/LinkedIn/ TikTok app registrations, so users authorise **GHL's** apps rather than an Evoke app — which is what makes publishing work without Evoke completing platform app review.

`evoke-oauth-callback` (GET) — agency app install

```

GHL OAuth Callback Webhook
├── GHL Extract Auth Code
│   └── GHL Exchange Code → services.leadconnectorhq.com/oauth/token
│       └── GHL Shape Install Tokens
│           └── GHL Store Install Tokens ← writes to workflow static data
│               └── GHL Install Respond

```

Run **once** when installing the Marketplace app on the agency. It seeds the refresh token that every other GHL flow depends on.

`evoke-ghl-token` (POST) — token broker

```

GHL Token Webhook
├── GHL Read Stored Tokens ← from $getWorkflowStaticData('global')
│   └── GHL Decide Refresh
│       └── GHL Token Expired? (if)
│           ├── true → GHL Refresh Agency Token → Shape → Persist Rotated Token → Respond
│           └── false → GHL Token Cached → Respond

```

Called **internally by the other GHL flows** (n8n calling its own webhook) so token refresh logic lives in exactly one place. If no token is stored it returns a readable error:

"No agency token stored. Install the Marketplace app on the agency first — that redirects through `/webhook/evoke-oauth-callback` and seeds it."

evoke-location-create (POST) — provision a sub-account

```
Location Create Webhook
└─ Loc Get Agency Token → calls /webhook/evoke-ghl-token
  └─ Loc Create Sub-Account → POST services.leadconnectorhq.com/locations/
    └─ Loc Extract Id → Loc Respond { locationId }
```

Each user should get their own GHL "location" — that isolation is what stops one user's connected pages being visible to another.

evoke-social-start (POST) — begin platform OAuth

```
Social Start Webhook
└─ Start Get Token
  └─ Start Build OAuth URL
    └─ Start Respond { success, startUrl, platform, locationId }
```

Builds:

```
https://services.leadconnectorhq.com/social-media-posting/oauth/{platform}/start
?locationId={locationId}&userId={uid}
```

Two deliberate safety properties, both worth preserving: - **The token is never returned.** The code node's comment is explicit: *"The token is deliberately NOT returned — this response reaches the browser."* - **Failures pass through as readable errors** rather than throwing, so the caller sees a reason instead of an empty response.

evoke-social-accounts (POST) — list / attach pages

```
Social Accounts Webhook
└─ Acct Get Token
  └─ Acct Is Attach? (if)
    ├── true → Acct Attach Page → Acct Normalize → Acct Respond
    └── false → Acct List Pages → (same)
```

`GET /social-media-posting/{locationId}/accounts` returns every page on the workspace. The frontend filters expired pages and attempts to scope them to the account that just signed in (see `ListConnectedPages` in `ghlService.js`).

evoke-social-post (POST) — publish

```
Social Post Webhook
└─ Post Get Token
  └─ Post Fetch Staff → GET /users/?locationId=...
    └─ Post Fetch Users → GET /social-media-posting/{locationId}/...
      └─ Post Build Payload (code)
        └─ Post Payload OK? (if)
          └─ Post Publish → POST /social-media-posting/{locationId}/posts
            └─ Post Normalize → Post Respond
```

`Post Build Payload` contains the most defensive code in the workflow. GHL **requires** a non-empty `userId`, and its field naming for that value is undocumented and has changed before. So the node resolves it in three escalating stages:

1. `body.userId` from the request
2. `ghlUserId` from workflow static data
3. **A recursive hunt** (max depth 6) over the entire GHL response for any key matching `userId|createdBy|created_by|authorId` — or a `users[]` array — holding a Mongo-style id (`/^[a-f0-9]{20,32}$/i`)

Request body:

```
{
  "uid": "sso_12345",
  "locationId": "...",
  "accountIds": [...],
  "summary": "the caption",
  "mediaUrls": ["https://..."],
  "scheduleDate": null
}
```

Omit `scheduleDate` to publish immediately.

`evoke-set-pit` (POST) — store the Private Integration Token

Admin-only. Writes the PIT (and optionally `userId`, `clientId`, `clientSecret`, `locationId`) into workflow static data.

Two good properties: - The token **never appears in a workflow export** — it lives in n8n's runtime storage, not the JSON. - The response echoes only the **last 6 characters** (`tokenEndsWith`), never the full value.

⚠ **Operational constraint:** n8n static data only persists on *production* runs. The workflow must be **ACTIVE** — values set during a manual test execution are lost.

6. Paid ads (5 webhooks)

Google Ads

`google-ads-oauth`

```
Extract Params → Exchange Code (oauth2.googleapis.com/token)
  → Get Profile (googleapis.com/oauth2/v2/userinfo)
  → List Accessible Customers (googleads.googleapis.com/v23/customers:listAccessibleCustomers)
  → Build Response → Respond
```

`google-ads-create-campaign` — five sequential `:mutate` calls on Google Ads API **v23**, each depending on the previous one's resource name:

```
1. campaignBudgets:mutate → budget
2. campaigns:mutate       → campaign
3. adGroups:mutate        → ad group
4. adGroupCriteria:mutate → keywords
5. adGroupAds:mutate      → the ad
```

`google-ads-metrics` — currently a `GAds Metrics Compute` code node feeding straight into a response. It does **not** call the Google Ads reporting API. See [Part 8](#).

Meta Ads

`meta-ads-oauth`

```
Extract Params → Exchange Code → Get Long-lived Token
  → Get Ad Accounts (/me/adaccounts) → Build Response → Respond
```

`meta-ads-boost` — four sequential creates on `act_{adAccountId}`, in Meta's required order:

```
1. /campaigns → campaign
2. /adsets    → ad set (targeting + budget)
3. /adcreatives → creative
4. /ads       → the ad
```

Both create-flows have a dedicated error-respond branch.

7. Messaging webhooks (3)

Webhook	Method	Purpose
<code>twilio-status-callback</code>	POST	Receives Twilio delivery status → <code>Format Twilio Status Event</code> → respond
<code>meta-whatsapp-webhook</code>	GET	Meta's subscription verification handshake → <code>Check Meta Verify Token</code> (if) → success or failure response
<code>meta-whatsapp-webhook</code>	POST	Inbound WhatsApp message events → <code>Format Meta WhatsApp Event</code> → respond

The same path serves both GET and POST — that's Meta's required pattern: GET verifies the subscription once, POST delivers events thereafter.

8. The scheduled publisher — the most important flow

This is what makes scheduled campaigns work without a browser tab open.

```
Schedule Trigger — every 15 minutes —
└─ Query Due Content
    └─ POST firestore.googleapis.com/v1/projects/evoke-cmo-agent2/
        databases/(default)/documents:runQuery
        Auth: googleApi predefined credential
        structuredQuery:
            from: content_items (allDescendants: true)
            where: status == 'scheduled'
                  AND requiresApproval == false
                  AND scheduledAt <= now()
            limit: 25
    └─ Has Due Items? (if)
        └─ Split Due Items
            └─ Map Firestore Doc to Day Payload
                └─ Day Normalize & Flatten
                    └─ Day Has LinkedIn? → Day LinkedIn Post → /v2/ugcPosts
                    └─ Day Has Instagram? → Day IG Create Media → Day IG Publish Media
                    └─ Day Has Facebook? → Day Facebook Post → /{pageId}/photos
                    └─ Day Has WhatsApp? → Split Recipients → Twilio
                    └─ Day Has SMS? → Split Recipients → Twilio
                        └─ Merge Publish Results
                            └─ Mark Content Published
                                PATCH firestore.googleapis.com/v1/{docPath}
```

The three query conditions — and why each matters

Condition	Why
<code>status == 'scheduled'</code>	Drafts and already-published items are excluded
<code>requiresApproval == false</code>	Human approval gate. An item stays invisible to the cron until a person approves it in <code>/queue</code> (which sets this to <code>false</code>) or it was created by <code>scheduleCampaignDays</code> (which sets it <code>false</code> at creation, because the user explicitly launched a multi-day campaign)
<code>scheduledAt <= now()</code>	Only items whose time has arrived

`limit: 25` caps each run, so a large backlog drains over several cycles rather than hammering every platform API at once.

Write-back

`Mark Content Published` PATCHes the Firestore document so the item won't match the query on the next run. **This is the idempotency mechanism — without it every item would republish every 15 minutes.**

Shared publishing nodes

The `Day *` nodes are shared between the scheduler and the `evoke-cmo-day` webhook, so manual single-day publishing and cron publishing take exactly the same code path.

9. Credential storage strategy

Four different mechanisms are in use:

Mechanism	Used for	Notes
n8n workflow static data (<code>\$getWorkflowStaticData('global')</code>)	GHL refresh token, access token, companyId, PIT, userId, clientId/secret	Never appears in a workflow export. Only persists on production runs — the workflow must be ACTIVE
n8n predefined credentials	Google API (Firestore access)	The proper n8n way

Mechanism	Used for	Notes
Passed in the request body	Per-user Facebook/Instagram/LinkedIn/Gmail tokens	Read from Firestore by the frontend, sent to n8n per call
Hard-coded in node parameters	⚠ Gemini API key, Twilio account SID	Should be moved to credentials. See 08

The GHL static-data approach has an explicit rationale in the code:

"Tokens live in n8n workflow static data — no Google credential needed, which keeps us clear of the org policy blocking service account keys."

10. Operating the workflow

Importing into a fresh n8n instance

1. Create the n8n instance (cloud or self-hosted)
 2. **Import** `EVOKE-CMO-v21-GHL.json`
 3. Configure the **Google API credential** used by the Firestore nodes
 4. Replace the hard-coded API keys in the Gemini and Twilio nodes with proper credentials
 5. **Toggle the workflow ACTIVE** — required for both webhooks and static-data persistence
 6. Point the frontend at the new host — set `VITE_N8N_BASE` (or the individual `VITE_N8N*_WEBHOOK` vars) and update the hard-coded `N8N_BASE` in `src/services/ghlService.js`
 7. Re-install the GHL Marketplace app so `evoke-oauth-callback` seeds the agency token
 8. POST the PIT to `/webhook/evoke-set-pit`
- `src/config.js` carries an abbreviated version of these instructions in its header comment.

Where the host URL is configured in the frontend

Location	How
<code>src/config.js</code>	Via <code>VITE_N8N_BASE</code> / <code>VITE_N8N*_WEBHOOK</code> env vars — configurable
<code>src/services/ghlService.js</code>	Hard-coded: <code>const N8N_BASE = 'https://n8n-zvxi.srv1837606.hstgr.cloud/webhook'</code>
<code>vite.config.js</code>	Hard-coded as the <code>/n8n-webhook</code> dev proxy target

Changing hosts requires editing all three. Flagged in [Part 8](#).

Debugging

- n8n's **Executions** tab shows every run with full per-node input/output — the fastest way to diagnose a failed publish.
- The `evoke-cmo` flow responds *before* it publishes, so a 200 in the browser tells you **nothing** about whether the post went out. Always check Executions.
- The scheduler runs every 15 minutes; a scheduled post can appear to "do nothing" for up to 15 minutes and still be healthy.

Part 5 — Third-Party Integrations

Every external service the platform talks to, what it's used for, where the code lives, and what condition it's in.

Integration status at a glance

Service	Category	Status	Where the integration lives
Invoke SSO	Identity	● Live	<code>src/lib/session.js</code>
Firebase (Firestore/Auth/Storage)	Data	● Live	<code>src/firebase.js</code> , <code>src/services/*</code>
GoHighLevel	Social publishing	● Live (shared workspace)	<code>src/services/ghlService.js</code> + n8n
Meta Graph (Facebook)	Publishing	● Live	n8n
Meta Graph (Instagram)	Publishing	● Live	n8n
LinkedIn	Publishing	● Live	n8n
Twilio	WhatsApp + SMS	● Live	n8n
Gmail API	Email sending	● Live	n8n
Groq	AI text	● Live	<code>api/generate*.js</code> , <code>vite.config.js</code>
Google Gemini	AI text + images	● Live	n8n, <code>api/generate-banner.js</code> , <code>api/gemini-image.js</code>
ImgBB	Image hosting	● Live	n8n
Pollinations	AI images (free)	● Live	<code>api/generate-banner.js</code>
Eventbrite	Events	● Live	<code>api/eventbrite.js</code> + n8n
Google Ads	Paid ads	● Create works, metrics stubbed	n8n
Meta Ads	Paid ads	● Create works, reporting unclear	n8n
TikTok	Social	● OAuth only	n8n
OpenAI	AI images	● Optional, key-gated	<code>api/generate-banner.js</code>
Resend	Transactional email	● Code ready, key not set (returns 501)	<code>api/send-email.js</code>
Supabase	Data	● Schema exists, reverted from live path	<code>api/brand-kb.js</code> , <code>src/lib/supabaseClient.js</code>
Razorpay	Payments	● Script loads dynamically	<code>src/pages/Purchase.jsx</code>
Cloudinary	Media	● Keys present, usage limited	<code>.env</code>
Luma / Meetup	Events	● Branches exist in n8n	n8n
Stripe	Payments	● Links optional, likely unset	<code>src/components/PlanGate.jsx</code>
SEMrush / Ahrefs / Search Console	SEO data	● Not integrated	— (page says so in its own comment)
Social inbox source	Messaging	● Not integrated	— (UI labels threads as examples)

● live · ● partial / conditional · ● not wired

1. Evoke SSO — identity

The platform is a satellite of the Evoke Marketplace identity system. It has **no login of its own**.

Property	Value
Portal	<code>https://accounts.evokemarketplace.com</code> (override: <code>VITE_ACCOUNTS_URL</code>)
Session API	<code>{VITE_API_BASE_URL}/auth/session</code> , <code>/auth/signout</code>
Cookie	<code>evoke_user</code> , non-httpOnly, written at the parent domain <code>.evokemarketplace.com</code>
App user id	<code>'sso_' + profile.custID</code>

Cookie payload shape:

```
{
  "status": "success",
  "message": "...",
  "data": {
    "email": "...", "firstName": "...", "lastName": "...",
    "custID": 260417001, "role": 4,
    "token": "<JWT>", "walletAddress": "0x..."
  }
}
```

The cookie is non-httpOnly **by design** so any `*.evokemarketplace.com` subdomain can render logged-in UI without an extra API round-trip. The file header is explicit that this is for **UI rendering only** — gated actions should still verify the JWT server-side.

Cookie lifetime is derived from the JWT's `exp` claim, capped at 7 days (`computeEvokeUserCookieMaxAgeSeconds`).

`src/lib/session.js` mirrors the same file in `converters_frontend` and `evoke_auth` — keep the three in sync.

2. Firebase — the primary data layer

Project: `evoke-cmo-agent2` (visible in the n8n Firestore URLs).

Service	Used for
Firestore	All live application data — see Part 6
Auth	Custom-token sign-in only. Signs the user in as the same <code>sso_<custID></code> uid so a verifiable ID token exists for API routes
Storage	Generated media
firebase-admin	Server-side ID-token verification in <code>api/_lib/verifyUser.js</code>

Config comes entirely from `VITE_FIREBASE_*` env vars — nothing is hardcoded in `src/firebase.js`.

n8n accesses Firestore over the **REST API** (`firestore.googleapis.com/v1/...`) using an n8n `googleApi` predefined credential, not the client SDK.

⚠️ Firestore security rules are **not** in this repository. They are managed in the Firebase console and must be reviewed separately.

3. GoHighLevel — the social publishing engine

The most consequential integration. GHL holds the Meta / LinkedIn / TikTok app registrations, so users authorise **GHL's** apps. That is what lets the platform publish to real accounts without Evoke completing platform app review for each network.

Property	Value
API base	<code>https://services.leadconnectorhq.com</code>

Property	Value
Auth model	Agency OAuth token (refresh-rotated) or a Private Integration Token (PIT)
Platforms	<code>facebook</code> , <code>instagram</code> , <code>linkedin</code> , <code>tiktok</code> , <code>threads</code> , <code>google</code>
Frontend	<code>src/services/ghlService.js</code>
Backend	7 n8n webhooks — see Part 4

Intended vs. current architecture

Intended: every user gets their own GHJ "location" (sub-account). That isolation is what keeps one user's connected pages invisible to every other user.

Current: running on a **Private Integration Token**, which is issued *per workspace*. So all users currently share one workspace (`SHARED_LOCATION_ID`, default `ePB8lCVVTfERqNprfkc`).

`ensureGhlLocation()` already implements the correct model first and only falls back:

```
try { /* create a dedicated location via n8n */ }
catch { /* fall back to SHARED_LOCATION_ID */ }
```

The moment the agency app can provision locations, the dedicated path becomes live for every new user **with no code change**.

Two safety behaviours that must not be removed

1. **Trusted popup origins.** `openOAuthPopup` only accepts `postMessage` from `services.leadconnectorhq.com` and `app.gohighlevel.com`. Without this allow-list any page could forge a connection event.
2. **The `scoped` flag.** `listConnectedPages()` returns `scoped: true` only when the page list could be narrowed to the account that just signed in (matching `originId` against the popup's `accountId`). When it returns `false`, the caller sees every page on the workspace — **and must not auto-select one**, because on a shared workspace that would silently link another user's page. The fallback exists to keep connecting possible, not as the intended path, and logs a `console.warn` when taken.

Environment override

`VITE_GHL_LOCATION_ID` — set it to point an environment at a different workspace, or to `''` to force per-user provisioning. The default is deliberately hard-coded rather than env-only, because a missing env var previously disabled connecting silently with no diagnosable cause. It is only a workspace identifier; the token that can act on it lives server-side in n8n.

4. Meta — Facebook, Instagram, WhatsApp, Ads

Graph API **v21.0** throughout.

Product	Flow
Facebook Pages	OAuth → short-lived → long-lived token → <code>/me/accounts</code> . Publishing: <code>/photos</code> (image), <code>/videos</code> (video), <code>/feed</code> (text)
Instagram Business	Discovered from the linked FB Page, or direct IG Business Login. Publishing is two-step and required : <code>POST /{igBusinessId}/media</code> (create container) → <code>POST /{igBusinessId}/media_publish</code>
WhatsApp Business	Webhook verification (GET) + inbound events (POST) at <code>meta-whatsapp-webhook</code> . Outbound messages currently go through Twilio , not Meta directly
Meta Ads	OAuth → <code>/me/adaccounts</code> . Creation: campaign → ad set → creative → ad on <code>act_{adAccountId}</code>

Env: `VITE_META_APP_ID`, `VITE_META_APP_SECRET`, `META_ADS_APP_ID`, `META_ADS_APP_SECRET`, `META_ADS_ACCESS_TOKEN`, `META_ADS_AD_ACCOUNT_ID`, `META_whatsapp_access_token`, `VITE_FACEBOOK_PAGE_ID`, `VITE_FACEBOOK_PAGE_ACCESS_TOKEN`, `VITE_INSTAGRAM_BUSINESS_ACCOUNT_ID`, `VITE_INSTAGRAM_ACCESS_TOKEN`.

5. LinkedIn

Two API generations are used side by side:

Content type	API	Endpoint
Text post	v2	<code>POST /v2/ugcPosts</code>
Image post	v2	<code>POST /v2/assets?action=registerUpload</code> → upload → <code>POST /v2/ugcPosts</code>
Video post	REST (versioned)	<code>/rest/videos?action=initializeUpload</code> → upload bytes → <code>?action=finalizeUpload</code> → Wait → <code>POST /rest/posts</code>
Profile	v2	<code>GET /v2/userinfo</code>

Video is the longest path in the workflow — six HTTP calls plus two `Wait` nodes, because LinkedIn processes video asynchronously and rejects a post referencing an unprocessed asset.

Env: `VITE_LINKEDIN_CLIENT_ID` (secret held in n8n).

6. Twilio — WhatsApp & SMS

Property	Value
Endpoint	<code>https://api.twilio.com/2010-04-01/Accounts/{AccountSid}/Messages.json</code>
Used for	WhatsApp and SMS in both the campaign flow and the scheduler
Status callback	<code>POST /webhook/twilio-status-callback</code>

Both channels pass through a "Split Recipients" code node so a campaign fans out to a recipient list.

Env: `twilio_sid`, `twilio_auth_token` (note: both are misspelled in `.env` — see 08).

 The Twilio Account SID is currently **hard-coded into the n8n node URLs**.

7. AI providers

Groq — primary text generation

Model	Used for	Where
<code>llama-3.1-8b-instant</code>	Campaign copy packages (15-field JSON)	<code>api/generate-campaign.js</code> , <code>vite.config.js</code>
<code>llama-3.3-70b-versatile</code>	Specialist agents (9 types)	<code>vite.config.js</code> / <code>api/run-agent</code>
caller-specified	Generic proxy	<code>api/generate.js</code>

Endpoint: `https://api.groq.com/openai/v1/chat/completions` (OpenAI-compatible). Env: `VITE_GROQ_API_KEY` — **server-side only** despite the `VITE_` prefix; it is read via `process.env` in the Edge functions and `LoadEnv` in Vite, never bundled into the client.

Google Gemini — n8n text + all image generation

Model	Used for	Where
<code>gemini-2.0-flash</code>	Campaign content, agent responses, product descriptions	n8n
<code>gemini-2.0-flash-exp</code>	Image generation (<code>responseModalities: ['IMAGE','TEXT']</code>)	<code>api/generate-banner.js</code> , <code>api/gemini-image.js</code> , n8n

Env: `GEMINI_API_KEY` (Vercel).  In n8n the key is embedded directly in node URLs.

OpenAI — optional images

`api/generate-banner.js` with `provider: 'dalle'` requests `gpt-image-1` at 1536×1024, quality `high`, returning base64 directly. Env: `OPENAI_API_KEY`.

The dev-mode path in `vite.config.js` still requests `dall-e-3` at 1024×1024 and fetches the returned URL to convert it. The two environments differ.

Pollinations — free fallback images

`https://image.pollinations.ai/prompt/{prompt}?width&height&model=flux&noLogo=true&seed` No API key. Seed must be `Math.random() * 2147483647` — `Date.now()` overflows 32-bit and is rejected.

ImgBB — image hosting for social

n8n uploads generated images to `https://api.imgbb.com/1/upload` because Meta and LinkedIn require a publicly reachable URL, not base64. Env: `VITE_IMGBB_API_KEY`.

8. Google Ads

Property	Value
API version	v23
OAuth	<code>oauth2.googleapis.com/token</code> → <code>customers:listAccessibleCustomers</code>
Creation	5 sequential <code>:mutate</code> calls — budget → campaign → ad group → keywords → ad
Metrics	⚠ <code>GAds Metrics Compute</code> is a code node with no API call
Frontend	<code>src/services/googleAdsService.js</code>

Env: `GOOGLE ADS DEVELOPER TOKEN`, plus `client_id` / `client_secret`.

9. Eventbrite

Two independent paths to the same API — a Vercel proxy **and** an n8n flow.

Path	File	Actions
Vercel	<code>api/eventbrite.js</code>	<code>create_venue</code> , <code>create_event</code> , <code>create_ticket</code> , <code>publish_event</code>
n8n	<code>evoked-cmo</code> flow, <code>Eventbrite Create Event</code> node	Event creation as part of a campaign
OAuth	n8n <code>eventbrite-oauth</code>	Token exchange + <code>/v3/users/me/organizations/</code>

The Vercel proxy holds no credentials — the **caller supplies the access token**.

Env: `VITE_EVENTBRITE_CLIENT_ID`.

Related event platforms with branches present in the campaign flow: **Luma** (`api.lu.ma/v1/calendar/create-event`) and **Meetup**.

10. TikTok

OAuth only — token exchange (`open.tiktokapis.com/v2/oauth/token/`) and profile fetch (`/v2/user/info/?fields=display_name,open_id`). Publishing to TikTok goes through **GoHighLevel**, which lists `tiktok` in `GHL_PLATFORMS`.

`public/tiktokGYdTycqX1no9l7SYXZqzYwbyML5WqPQf.txt` is TikTok's domain-verification file.

Env: `VITE_TIKTOK_CLIENT_ID`, `VITE_TIKTOK_CLIENT_SECRET`, `VITE_TIKTOK_REDIRECT_URI`.

11. Email

Three separate email mechanisms coexist:

Mechanism	Path	Status
Gmail API	n8n: refresh token → build MIME → <code>messages/send</code>	🟢 Live — sends as the user
Resend	<code>api/send-email.js</code>	🟡 Code ready; returns 501 until <code>RESEND_API_KEY</code> is set
SMTP/other	—	Not present

Resend default sender: `RESEND_FROM_EMAIL` → `'EVOX AI CMO <onboarding@resend.dev>'`.

12. Payments

Provider	Where	Status
Razorpay	<code>src/pages/Purchase.jsx</code> loads the checkout script dynamically	Env: <code>VITE_RAZORPAY_KEY_ID</code>
Stripe	<code>src/components/PlanGate.jsx</code> — <code>STRIPE_LINKS</code> from <code>VITE_STRIPE_LINK_A/B/C</code>	Optional; when unset the gate falls back to navigating to <code>/requiredPlan</code>

There is also an internal **token economy** (`TOKEN_PACKAGES` in `userService.js`): Starter 10/\$9.99, Growth 20/\$17.99, Pro 35/\$24.99. One token is deducted per campaign launch via `deductToken()`, which throws `'Insufficient tokens'` at zero.

13. Supabase — built, then rolled back

A complete Postgres schema exists (`docs-internal/supabase-schema.sql`) with `user_profiles` and `brand_knowledge_base` tables, RLS policies, `updated_at` triggers and indexes.

It is not in the live path. The header of `src/services/knowledgeBaseService.js` records why, dated 2026-08-04:

Vercel API routes don't run under plain `vite` (only under `vercel dev` or a real Vercel deployment), and the required credentials (Firebase admin key, Supabase service-role key) were never added — so every save was silently failing with a 404.

What still exists and works: - `docs-internal/supabase-schema.sql` — the full schema - `api/brand-kb.js` + `api/_lib/*` — the working Node API route - `src/lib/apiAuth.js` — the authenticated fetch helper - `src/lib/supabaseClient.js` — the browser client - `/supabase-test` — a localhost-only connectivity check

To reinstate: add both credential sets, test under `vercel dev` or a real deployment (**not** plain `vite`), then point `knowledgeBaseService.js` back at `authedFetch`.

See [Part 6](#) for the schema.

14. EGT — Evoke Gratitude Token (blockchain)

An on-chain token balance surfaced in the UI.

File	Role
<code>src/components/EgtWalletHeader.jsx</code>	Header wallet display
<code>src/components/GratitudeToken.jsx</code>	Token display/award
<code>src/lib/evokeUserCookie.js</code>	Reads the wallet address from the SSO cookie

File	Role
<code>src/lib/egtRewardPoolBalance.js</code>	Reads the reward-pool balance via RPC
<code>src/lib/formatEgtBalance.js</code>	Display formatting

The wallet address arrives in the SSO cookie and is validated against `/^0x[a-fA-F0-9]{40}$/` before use. Env: `VITE_EGT_RPC_URL`, `VITE_EGT_REWARD_POOL_ADDRESS`.

15. Analytics & media

Service	Where	Notes
Google Ads gtag	<code>index.html</code> , <code>src/lib/gtag.js</code>	Conversion ID <code>AW-18246572299</code> ; helpers <code>gtagEvent</code> , <code>trackPurchase</code> , <code>trackSignup</code>
Cloudinary	<code>.env</code> only	Keys present (<code>CLOUDINARY_*</code> , <code>VITE_CLOUDINARY_CLOUD_NAME</code>); the <code>cloudinary</code> npm package is a dependency
QR Server API	<code>src/services/qrCodeService.js</code>	Free, no key. Overlays registration QR codes on event banners
WaveSpeed	<code>.env</code> only	<code>VITE_WAVESPEED_API_KEY</code> — appears twice; purpose undocumented

16. Not integrated — but presented in the UI

Two areas show data that has no live source. **Both are honestly labelled in the code**, and that labelling should be preserved until real integrations land.

Area	File	What the code says
SEO Intelligence Center	<code>src/pages/SeoIntelligenceCenterPage.jsx</code>	Its own header comment states that no SEMrush / Ahrefs / Search Console integration exists anywhere in the app, so Domain Authority, Organic Traffic, Keywords Ranked, Backlinks and the keyword table have zero real data source
Social Inbox	<code>src/pages/SocialInbox.jsx</code>	Renders an in-UI banner: <i>"Example conversations — no live LinkedIn/Instagram/Facebook/WhatsApp inbox is connected yet."</i> Replies are saved to the user's account, and "Suggested Reply" is a real AI call

Part 6 — Data Model

The platform runs on **Firebase Firestore**. A Supabase Postgres schema also exists but is currently **not** in the live path (see § Supabase).

Firebase project: `evoke-cmo-agent2` **Primary key everywhere:** `uid = 'sso_' + custID` (from `src/lib/authUtils.js`)

Firestore collection map

<code>users/{uid}</code>	← the central document
<code>contacts/{contactId}</code>	← CRM
<code>teamMembers/{memberId}</code>	← Team
<code>teamInvites/{inviteId}</code>	← Team invitations
<code>inboxThreads/{threadId}</code>	← Social Inbox reply history
<code>content_items/{itemId}</code>	← top-level: all generated content
<code>governance_audit_log/{entryId}</code>	← top-level: brand governance decisions

Two design decisions worth understanding:

1. **The Brand Knowledge Base is a nested field on `users/{uid}`, not a subcollection.** `knowledgeBaseService.js` states this is deliberate — it avoids subcollection permission issues.
2. **`content_items` is top-level, not nested under the user.** It has to be — the n8n scheduler queries it across all users with `allDescendants: true`, which a per-user subcollection would make far more awkward.

`users/{uid}` — the central document

Written by `src/services/userService.js` and `src/services/knowledgeBaseService.js`.

Identity & profile

Field	Type	Set by	Notes
<code>displayName</code>	string	<code>getOrCreateUser</code>	From the SSO profile
<code>email</code>	string	<code>getOrCreateUser</code>	From the SSO profile
<code>createdAt</code>	timestamp	<code>getOrCreateUser</code>	<code>serverTimestamp()</code>

Plan & billing

Field	Type	Set by	Notes
<code>userPlan</code>	string	<code>saveSelectedPlan</code> , <code>OnboardingWizard</code>	<code>free \ package-a \ package-b \ package-c</code>
<code>selectedPlan</code>	string	same	Duplicate of <code>userPlan</code> — both are written together
<code>trialStartedAt</code>	timestamp	—	Read by <code>useUserPlan</code> to compute <code>trialDaysLeft</code> against 14 days
<code>tokenBalance</code>	number	<code>addTokens</code> , <code>deductToken</code>	Defaults 0. One token per campaign launch

`useUserPlan` reads `data?.userPlan || data?.selectedPlan || 'free'`, so either field works. The duplication is historical — see 08.

Onboarding state

Field	Type	Notes
<code>onboardingComplete</code>	boolean	Set <code>true</code> by <code>saveOnboardingData</code>

Field	Type	Notes
<code>onboardingData</code>	object	<code>{ background, industry, goal }</code>
<code>onboardingCompletedAt</code>	timestamp	
<code>chatOnboardingDone</code>	boolean	Legacy chat flow
<code>chatOnboardingCompletedAt</code>	timestamp	Legacy chat flow
<code>brandSetupComplete</code>	boolean	Written by <code>OnboardingWizard</code>
<code>recommendedRoutes</code>	string[]	Personalised route suggestions
<code>productTourSeen</code>	boolean	Set by <code>markTourSeen</code> so the tour doesn't replay

`socialAccounts` (nested map)

Initialised by `getOrCreateUser`, updated by `saveSocialAccount(uid, platform, data)`.

```
socialAccounts: {
  facebook: { connected, pageId, pageAccessToken, pageName },
  instagram: { connected, businessAccountId, pageName },
  linkedin: { connected, personUrn, accessToken, name },
  twitter: { connected, accessToken, username, userId },
  whatsapp: { connected, phoneNumberId, accessToken },
  gmail: { connected, email },
}
```

Platforms connected via GoHighLevel carry a different shape under the same key:

```
socialAccounts: {
  ghl: { connected, locationId, dedicated }, // the workspace itself
  facebook: { connected, viaGhl: true, ghlAccountId, locationId, pageName, avatar },
  // ...same for instagram, linkedin, tiktok, threads, google
}
```

`ghl.dedicated` is `true` when the user has their own provisioned location, `false` when they're on the shared workspace.

`disconnectSocialAccount` replaces the whole platform object with `{ connected: false }`, clearing the stored tokens.

`knowledgeBase` (nested map) — the Brand Knowledge Base

Written by `saveKnowledgeBase(uid, data)` and `appendJourneyOutput(uid, step, summary)`.

```
knowledgeBase: {
  companyName, industry, audienceType, primaryObjective,
  description, tone, challenge, platforms, // ...wizard answers
  updatedAt: <timestamp>,

  journeyOutputs: {
    strategy: { summary, updatedAt },
    audience: { summary, updatedAt },
    // ...one entry per completed journey step
  }
}
```

`journeyOutputs` is the mechanism that lets later agents see what earlier agents produced. Keys match the step names in `src/lib/journeyHandoff.js`.

`content_items/{itemId}` — generated content

The most operationally important collection. Written by `src/services/contentService.js`; read by `/queue`, `/results` and — critically — the **n8n 15-minute scheduler**.

Core fields

Field	Type	Values / notes
<code>userId</code>	string	The owning <code>sso_<custID></code> uid
<code>campaignId</code>	string	Groups every item generated in one run
<code>campaignName</code>	string	
<code>campaignType</code>	string	<code>event \</code> <code>product \</code> <code>growth_strategy \</code> <code>post-content \</code> <code>...</code>
<code>type</code>	string	<code>post \</code> <code>email \</code> <code>ad \</code> <code>seo \</code> <code>strategy</code>
<code>platform</code>	string	<code>linkedin \</code> <code>instagram \</code> <code>facebook \</code> <code>tiktok \</code> <code>whatsapp \</code> <code>email \</code> <code>gmail \</code> <code>twitter \</code> <code>' '</code>
<code>text</code>	string	The caption / body
<code>subject</code>	string	Email subject (email items only)
<code>data</code>	string	JSON payload for structured items (e.g. strategy docs)
<code>imageUrl</code> , <code>videoUrl</code>	string	Media
<code>source</code>	string	<code>campaign \</code> <code>post-content</code>
<code>createdAt</code> , <code>updatedAt</code>	timestamp	

Workflow fields — read these carefully

Field	Type	Meaning
<code>status</code>	string	<code>draft \</code> <code>approved \</code> <code>scheduled \</code> <code>published \</code> <code>rejected</code>
<code>requiresApproval</code>	boolean	<code>true</code> = stays in <code>/queue</code> for manual review. <code>false</code> = eligible for n8n auto-publish
<code>scheduledAt</code>	timestamp ISO string null	The real trigger the n8n Firestore query filters on
<code>publishedAt</code>	timestamp null	Set when status becomes <code>published</code>

The lifecycle



The multi-day campaign shortcut: `scheduleCampaignDays()` writes days 2..N directly at `status: 'scheduled'` + `requiresApproval: false`, skipping the approval queue — the user explicitly launched the campaign, so each day doesn't need re-approving.

Extra fields on scheduled-day rows

`scheduleCampaignDays` writes a richer document, because the n8n scheduler must be able to publish it **without any browser involvement**:

Field	Purpose
<code>day</code>	Which campaign day this is
<code>dailySchedule</code>	Per-day posting schedule array
<code>linkedinPost</code> , <code>instagramCaption</code> , <code>facebookPost</code> , <code>whatsappMessage</code>	Per-platform copy
<code>emailSubject</code> , <code>emailBody</code>	Email content
<code>imageUrl</code>	Media
<code>whatsappRecipients</code> , <code>emailRecipients</code>	Recipient lists
<code>userCredentials</code>	⚠️ The platform tokens needed to post — see below

⚠️ `userCredentials` stores access tokens inside the content document so the headless cron can publish without a session. This is what makes scheduled posting work, and it is also the single most sensitive field in the database. Flagged in [Part 8](#).

Query note

`getContentItems()` sorts **client-side**:

```
.sort((a, b) => (b.createdAt?.seconds || 0) - (a.createdAt?.seconds || 0))
```

This is deliberate — it avoids needing a composite Firestore index on `(userId, status, createdAt)`. It also means the whole matching set is fetched before sorting, so it will not scale to very large libraries without pagination.

`users/{uid}/contacts/{contactId}` — CRM

Written by `src/services/crmService.js`.

Field	Type	Values
<code>name</code>	string	
<code>email</code>	string	
<code>company</code>	string	
<code>phone</code>	string	
<code>stage</code>	string	<code>lead \</code> <code>prospect \</code> <code>customer \</code> <code>retained</code>
<code>score</code>	number	0–100, auto-calculated by <code>calcScore()</code>
<code>notes</code>	string	
<code>tags</code>	string[]	
<code>source</code>	string	<code>manual \</code> <code>campaign \</code> <code>import</code>
<code>lastContact</code>	timestamp null	
<code>createdAt</code>	timestamp	

Stage lifecycle automation: moving a contact forward a stage (`lead` → `prospect` → `customer` → `retained`) automatically sends a templated email, if the contact has an address on file. The service's own comment states the rule:

Failures are logged, never thrown — automation should never block a save.

`users/{uid}/teamMembers/{memberId}` — Team

Written by `src/services/teamService.js`.

Field	Type	Values
<code>name</code>	string	
<code>email</code>	string	
<code>role</code>	string	<code>owner \ admin \ editor \ viewer</code>
<code>status</code>	string	<code>active \ inactive</code>
<code>joinedAt</code>	timestamp	

`ensureOwnerMember()` guarantees the account holder always exists as `owner`.

`users/{uid}/teamInvites/{inviteId}` — Invitations

Field	Type	Values
<code>email</code>	string	
<code>role</code>	string	<code>admin \ editor \ viewer</code> (never <code>owner</code>)
<code>sentAt</code>	timestamp	

Invites are sent via `sendEmail()` → `/api/send-email` (Resend).

`users/{uid}/inboxThreads/{threadId}` — Social Inbox replies

Written by `src/services/socialInboxService.js`.

Field	Type	Notes
<code>thread</code>	array	<code>{ from: 'me' \ 'them', text, time }[]</code>
<code>updatedAt</code>	timestamp	

This collection stores **replies only**. There is no live social-platform message source (no Meta Conversations API, no GHL inbox integration). `SocialInbox.jsx` shows example conversations as the base data; this collection layers the user's real replies on top so they survive a reload.

`governance_audit_log/{entryId}` — Brand Governance audit trail

Top-level collection, written by `src/services/governanceService.js`. Every conformance review decision is appended so the audit log survives a reload rather than living only in React state.

Field	Type	Notes
<code>userId</code>	string	Owner
<code>label</code>	string	From <code>entry.id</code>
<code>type</code>	string	Review type
<code>status</code>	string	Defaults <code>'flagged'</code>

Field	Type	Notes
score	number	Defaults 0
agent	string	Defaults 'Brand Governance Agent'
createdAt	timestamp	

Firestore security rules

⚠ **Not in this repository.** Rules are managed in the Firebase console and must be reviewed there. Given that `content_items` and `governance_audit_log` are **top-level collections keyed by a `userId` field**, the rules must enforce `userId` matching on both — the collection structure alone does not isolate users.

Supabase Postgres schema (not live)

Source: `docs-internal/supabase-schema.sql`. Idempotent — safe to re-run in full.

user_profiles

```
create table if not exists user_profiles (
  id                uuid primary key,      -- deterministic UUID from 'sso_<custID>'
  sso_uid           text unique not null,  -- the original 'sso_12345' string
  email             text,
  display_name      text,
  plan              text not null default 'free',
  onboarding_complete boolean not null default false,
  onboarding_data    jsonb,
  brand_kb_complete boolean not null default false,
  brand_setup_complete boolean not null default false,
  brand_name         text,
  recommended_routes text[],
  token_balance      integer not null default 0,
  social_accounts    jsonb not null default '{}'::jsonb,
  created_at         timestampz not null default now(),
  updated_at         timestampz not null default now()
);
create index if not exists idx_user_profiles_sso_uid on user_profiles (sso_uid);
```

brand_knowledge_base

```
create table if not exists brand_knowledge_base (
  user_id           uuid primary key references user_profiles(id) on delete cascade,
  business_name     text,
  industry           text,
  description        text,
  audience          text,
  goal              text,
  platforms         text[],
  tone              text,
  challenge          text,
  journey_outputs    jsonb not null default '{}'::jsonb, -- mirrors appendJourneyOutput()
  completed_at      timestampz,
  updated_at        timestampz not null default now()
);
```

Triggers

A shared `set_updated_at()` trigger function fires `before update` on both tables.

Row Level Security

```
alter table user_profiles      enable row level security;
alter table brand_knowledge_base enable row level security;

create policy "own profile" on user_profiles
  for all using (auth.uid() = id)      with check (auth.uid() = id);

create policy "own brand kb" on brand_knowledge_base
  for all using (auth.uid() = user_id) with check (auth.uid() = user_id);
```

`auth.uid()` only resolves when the request carries a **Supabase-signed JWT**. The design was for `api/supabase-session.js` to mint that JWT after verifying the real Evoke SSO session server-side. **That file does not exist in the repository** — it is referenced by the schema comments but was never committed (or was removed during the rollback). Anyone reinstating Supabase must write it.

Firestore → Supabase field mapping

Firestore (<code>users/{uid}</code>)	Supabase (<code>user_profiles</code>)
document id <code>sso_<custID></code>	<code>sso_uid</code> (text) + <code>id</code> (derived UUID)
<code>userPlan</code> / <code>selectedPlan</code>	<code>plan</code> (single field — the duplication is resolved)
<code>onboardingComplete</code>	<code>onboarding_complete</code>
<code>onboardingData</code>	<code>onboarding_data</code> (jsonb)
<code>brandSetupComplete</code>	<code>brand_setup_complete</code>
<code>recommendedRoutes</code>	<code>recommended_routes</code> (text[])
<code>tokenBalance</code>	<code>token_balance</code>
<code>socialAccounts</code>	<code>social_accounts</code> (jsonb)
<code>knowledgeBase.*</code>	→ the <code>brand_knowledge_base</code> table
<code>knowledgeBase.journeyOutputs</code>	<code>journey_outputs</code> (jsonb)

Not covered by the Supabase schema: `content_items`, `contacts`, `teamMembers`, `teamInvites`, `inboxThreads`, `governance_audit_log`. A full migration would need tables for all of these, and the n8n scheduler's Firestore `runQuery` would have to be rewritten against Postgres.

Part 7 — Environment & Deployment

1. Prerequisites

Requirement	Notes
Node.js 18+	Required for the Vite 5 toolchain and native <code>fetch</code>
npm	<code>package-lock.json</code> is committed — use npm, not yarn/pnpm
A populated <code>.env</code>	Not in git. Obtain from the current maintainer or the Vercel dashboard
Vercel CLI (optional)	Only needed to exercise the real <code>api/</code> functions locally

2. Local development

```
npm install
```

```
npm run dev
```

The app runs at `http://localhost:3007`.

The port comes from `package.json` (`vite --port 3007`), which overrides `server.port: 5173` in `vite.config.js`. `.claude/launch.json` also pins 3007 with `--strictPort`. If the port is busy you likely have a stale dev server — kill it rather than letting Vite pick another port, because several OAuth redirect URIs are registered against 3007.

What `npm run dev` gives you

- ✓ The full React app
- ✓ `/api/generate`, `/api/generate-campaign`, `/api/generate-banner`, `/api/eventbrite`
- ✓ `/api/run-agent` (dev-only — no production equivalent)
- ✓ CORS proxies to n8n, Groq, the Evoke API and Pollinations
- ✓ Auto sign-in as the hardcoded dev profile
- ✓ All plan gates bypassed
- ✗ `/api/send-email`, `/api/gemini-image`, `/api/brand-kb` — these 404 locally

Running the real Vercel functions locally

```
vercel dev
```

Required if you are working on `api/send-email.js`, `api/gemini-image.js` or `api/brand-kb.js`. Plain `vite` will not run them — that is precisely how the Supabase Brand-KB migration silently failed and had to be reverted.

Other scripts

```
npm run build
```

Vite production build → `dist/` (with `--emptyOutDir`).

```
npm run preview
```

Serves the built `dist/` locally.

```
npm run lint
```

ESLint over `js,jsx` with `--max-warnings 0`.

⚠ There is **no ESLint config file** in the repository, so this script will not currently run as-is. Noted in [Part 8](#).

3. Environment variables

`.env` lives at the repo root and is **git-ignored**. No values appear in this document — only names.

Understanding the prefix rule matters here:

- **VITE_ -prefixed** vars are inlined into the **client bundle** by Vite and are therefore **publicly visible** in the browser. Only put non-secret values here.
- **Un-prefixed** vars are available server-side only (`process.env` in Vercel functions; `loadEnv(mode, cwd, '')` in `vite.config.js`, which loads *everything*).

⚠ Several genuine secrets currently carry a `VITE_` prefix. Read the notes column carefully — and see [Part 8](#).

Firebase (client)

Variable	Purpose
<code>VITE_FIREBASE_API_KEY</code>	Firebase web API key (public by design)
<code>VITE_FIREBASE_AUTH_DOMAIN</code>	Auth domain
<code>VITE_FIREBASE_PROJECT_ID</code>	<code>evoke-cmo-agent2</code>
<code>VITE_FIREBASE_STORAGE_BUCKET</code>	Storage bucket
<code>VITE_FIREBASE_MESSAGING_SENDER_ID</code>	FCM sender id
<code>VITE_FIREBASE_APP_ID</code>	App id
<code>VITE_FIREBASE_MEASUREMENT_ID</code>	Analytics id

Firebase Admin (server — Vercel only)

Variable	Purpose
<code>FIREBASE_ADMIN_PROJECT_ID</code>	Service-account project
<code>FIREBASE_ADMIN_CLIENT_EMAIL</code>	Service-account email
<code>FIREBASE_ADMIN_PRIVATE_KEY</code>	Private key — store with escaped <code>\n</code> ; the code unescapes at runtime

⚠ **Not currently set.** This is one of the two reasons the Supabase Brand-KB route was rolled back.

Evoke SSO & platform

Variable	Purpose
<code>VITE_ACCOUNTS_URL</code>	Accounts portal (default <code>https://accounts.evokemarketplace.com</code>)
<code>VITE_API_BASE_URL</code>	Evoke backend for <code>/auth/session</code> and <code>/auth/signout</code>
<code>VITE_COOKIE_DOMAIN</code>	Cookie domain, e.g. <code>.evokemarketplace.com</code> . Set <code>none</code> /empty for localhost
<code>VITE_MARKETPLACE_URL</code>	Marketplace link target
<code>VITE_EVOKE_API_KEY</code>	Evoke API key

n8n

Variable	Purpose
<code>VITE_N8N_BASE</code>	Base host for all n8n webhooks
<code>VITE_N8N_WEBHOOK</code>	Full override for the main campaign webhook
<code>VITE_N8N_DAY_WEBHOOK</code>	Full override for the day webhook
<code>VITE_N8N_AGENT_WEBHOOK</code>	Full override for the agents webhook

AI providers (server-side)

Variable	Purpose	Notes
<code>VITE_GROQ_API_KEY</code>	Groq API key	⚠️ <code>VITE_</code> -prefixed but only read server-side. The prefix is misleading — rename to <code>GROQ_API_KEY</code>
<code>GEMINI_API_KEY</code>	Gemini (images)	Correctly un-prefixed
<code>OPENAI_API_KEY</code>	OpenAI images	Correctly un-prefixed
<code>VITE_IMGBB_API_KEY</code>	ImgBB image hosting	
<code>VITE_WAVESPEED_API_KEY</code>	WaveSpeed	Appears twice in <code>.env</code> ; purpose undocumented

GoHighLevel

Variable	Purpose
<code>GHL_CLIENT_ID</code>	Marketplace app client id
<code>GHL_CLIENT_SECRET</code>	Marketplace app secret
<code>GHL_PIT_token</code>	Private Integration Token (<i>note the typo in the name</i>)
<code>VITE_GHL_LOCATION_ID</code>	Shared workspace id. Appears twice in <code>.env</code> . Set to <code>' '</code> to force per-user provisioning

Meta / Facebook / Instagram

Variable	Purpose
<code>VITE_META_APP_ID</code> / <code>VITE_META_APP_SECRET</code>	Meta app credentials
<code>Vite_facebook_secret</code>	Facebook secret (<i>inconsistent casing</i>)
<code>VITE_FACEBOOK_PAGE_ID</code> / <code>VITE_FB_PAGE_ID</code>	Page id — two vars for the same thing
<code>VITE_FACEBOOK_PAGE_ACCESS_TOKEN</code> / <code>VITE_FB_PAGE_ACCESS_TOKEN</code>	Page token — two vars for the same thing
<code>VITE_INSTAGRAM_BUSINESS_ACCOUNT_ID</code> / <code>VITE_INSTAGRAM_ID</code>	IG business account — two vars
<code>VITE_INSTAGRAM_ACCESS_TOKEN</code>	IG token
<code>META_ADS_APP_ID</code> / <code>META_ADS_APP_SECRET</code>	Meta Ads app
<code>META_ADS_ACCESS_TOKEN</code> / <code>META_ADS_AD_ACCOUNT_ID</code>	Meta Ads account
<code>VITE_META_AD_ACCOUNT_ID</code>	⚠️ Duplicate of <code>META_ADS_AD_ACCOUNT_ID</code>
<code>META_whatsapp_access_token</code>	WhatsApp Business token
<code>VITE_META_API_BASE</code>	Meta/Evoke FastAPI backend base URL

Other platforms

Variable	Purpose
<code>VITE_LINKEDIN_CLIENT_ID</code>	LinkedIn app
<code>VITE_TIKTOK_CLIENT_ID</code> / <code>VITE_TIKTOK_CLIENT_SECRET</code> / <code>VITE_TIKTOK_REDIRECT_URI</code>	TikTok app ⚠️ the secret must not be <code>VITE_</code> -prefixed
<code>VITE_TWITTER_CLIENT_ID</code>	Twitter/X app
<code>VITE_EVENTBRITE_CLIENT_ID</code>	Eventbrite app
<code>GOOGLE_ADS_DEVELOPER_TOKEN</code>	Google Ads developer token
<code>client_id</code> / <code>client_secret</code>	Generic Google OAuth (<i>non-descriptive names</i>)
<code>twilo_sid</code> / <code>twilo_auth_token</code>	Twilio (<i>both misspelled</i> — "twilo")

Storage & payments

Variable	Purpose
<code>CLOUDINARY_CLOUD_NAME</code> / <code>CLOUDINARY_API_KEY</code> / <code>CLOUDINARY_API_SECRET</code>	Cloudinary (server)
<code>VITE_CLOUDINARY_CLOUD_NAME</code>	Cloudinary (client)
<code>VITE_RAZORPAY_KEY_ID</code>	Razorpay checkout
<code>VITE_STRIPE_LINK_A</code> / <code>_B</code> / <code>_C</code>	Stripe payment links per plan. Referenced in <code>PlanGate.jsx</code> but not present in <code>.env</code> — the gate falls back to <code>{plan}</code>

Supabase

Variable	Purpose
<code>VITE_SUPABASE_URL</code>	Project URL
<code>VITE_SUPABASE_ANON_KEY</code>	Anon key (public by design, RLS-protected)
<code>SUPABASE_SERVICE_ROLE_KEY</code>	⚠️ Bypasses RLS. Server-only. Not currently set — the other reason the Brand-KB route was rolled back

EGT (blockchain)

Variable	Purpose
<code>VITE_EGT_RPC_URL</code>	RPC endpoint
<code>VITE_EGT_REWARD_POOL_ADDRESS</code>	Reward pool contract address

Email

Variable	Purpose
<code>RESEND_API_KEY</code>	Resend. Not set → <code>/api/send-email</code> returns 501
<code>RESEND_FROM_EMAIL</code>	Default sender
<code>VITE_ADMIN_EMAIL</code>	Approval-email recipient

⚠️ `.env` **currently contains a stray line beginning** `For ...` **which is not a valid variable assignment.** It parses as a var named `For`. Remove it.

4. Deployment — frontend (Vercel)

Configuration

`vercel.json`:

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

The catch-all rewrite makes client-side routing work. Vercel resolves `/api/*` against the `api/` directory **before** applying rewrites, so the functions remain reachable.

`.vercel/project.json` holds the linked project id.

Steps

1. Connect the Git repository to the Vercel project (or `vercel link`)
2. Add **every** environment variable in §3 to the Vercel dashboard — including the un-prefixed server-side ones
3. Push to the deployment branch; Vercel builds automatically
4. Verify the custom domain resolves and that `VITE_COOKIE_DOMAIN` matches it — SSO breaks if the app is not served from a `*.evokemarketplace.com` subdomain, because the `evoke_user` cookie is scoped to that parent domain

Runtime split

Route	Runtime	Why
<code>api/generate.js</code> , <code>generate-campaign.js</code> , <code>generate-banner.js</code> , <code>gemini-image.js</code> , <code>eventbrite.js</code> , <code>send-email.js</code>	Edge	Declared <code>export const config = { runtime: 'edge' }</code>
<code>api/brand-kb.js</code>	Node	Needs <code>firebase-admin</code> , which doesn't run on Edge

5. Deployment — n8n

The workflow is **not** deployed from this repository. Full procedure in [Part 4 § 10](#). Summary:

1. Import `EVOKE-CMO-v21-GHL.json`
2. Configure the Google API credential (Firestore access)
3. Replace hard-coded API keys in the Gemini and Twilio nodes with proper credentials
4. **Activate the workflow** — required for webhooks *and* for static-data persistence
5. Update the n8n host in **three** frontend places: - `VITE_N8N_BASE` env var (`src/config.js` reads it) - the hard-coded `N8N_BASE` in `src/services/ghlService.js` - the `/n8n-webhook` proxy target in `vite.config.js`
6. Re-install the GHL Marketplace app so `evoke-oauth-callback` seeds the agency token
7. POST the PIT to `/webhook/evoke-set-pit`

6. Firebase setup

Item	Where
Firestore database	Firebase console — project <code>evoke-cmo-agent2</code>
Security rules	⚠️ Console-managed, not in this repo . Must be reviewed separately
Auth	Custom-token sign-in must be enabled
Storage	Bucket per <code>VITE_FIREBASE_STORAGE_BUCKET</code>

Item	Where
Service account	Needed for <code>FIREBASE_ADMIN_*</code> and for the n8n <code>googleApi</code> credential

7. Post-deployment verification checklist

#	Check	How
1	SSO sign-in works	Visit the app signed out → should redirect to the accounts portal and return authenticated
2	The cookie is readable	DevTools → Application → Cookies → <code>evoke_user</code> present on the app's domain
3	Plan gates enforce	Sign in on a <code>free</code> plan → a <code>package-b</code> route should show the lock card, not the page
4	AI generation works	Run a campaign → <code>POST /api/generate*</code> returns 200
5	n8n receives the campaign	n8n → Executions → a new <code>evoke-cmo</code> run appears
6	Social connect works	<code>/connect-accounts</code> → connect a platform → the OAuth popup completes and the page is listed
7	Immediate publish works	<code>/post-content</code> → publish → the post appears on the real account
8	The scheduler runs	Schedule an item for +20 min → confirm it publishes and <code>status</code> flips to <code>published</code> in Firestore
9	Dev routes are hidden	<code>/dev-reset</code> and <code>/supabase-test</code> redirect to <code>/</code> in production
10	Email sends	Only after <code>RESEND_API_KEY</code> is set — otherwise expect a 501

Step 8 is the one most worth doing properly. It is the only end-to-end proof that the cron, the Firestore query, the platform tokens and the write-back are all working together.

8. Troubleshooting

Symptom	Likely cause
<code>/api/send-email</code> , <code>/api/gemini-image</code> or <code>/api/brand-kb</code> 404s locally	Expected — plain <code>vite</code> doesn't run <code>api/</code> . Use <code>vercel dev</code>
Campaign returns 200 but nothing publishes	Normal — <code>evoke-cmo</code> responds before publishing. Check n8n → Executions
Scheduled post never publishes	Check all three query conditions in Firestore: <code>status == 'scheduled'</code> , <code>requiresApproval == false</code> , <code>scheduledAt <= now()</code>
Scheduled post publishes repeatedly	The <code>Mark Content Published</code> write-back is failing — that node is the idempotency mechanism
GHL calls fail with "No agency token stored"	The Marketplace app isn't installed. Re-run the <code>evoke-oauth-callback</code> install flow
PIT stored but not persisting	The n8n workflow isn't ACTIVE — static data only persists on production runs
Social connect links the wrong page	<code>listConnectedPages</code> returned <code>scoped: false</code> on the shared workspace. Check the <code>console.warn</code>
Everything is unlocked and you're always logged in	You're on <code>localhost</code> — all three dev bypasses are active
Plan gates don't enforce in production	Check <code>PLAN_GATES_ENABLED</code> in <code>PlanGate.jsx</code> (currently <code>true</code>)
Pollinations rejects the request	Seed exceeded 32-bit int — must be <code>Math.random() * 2147483647</code>
<code>npm run lint</code> fails to start	No ESLint config exists in the repo

Part 8 — Known Gaps & Risks

An honest assessment of the system's current state, written for handover. Everything here was verified against the code on **5 August 2026** — nothing is speculative.

No secret values appear in this document. Where a credential is exposed, the location is named and the value is not reproduced.

Severity summary

Severity	Count	Theme
● High	6	Credentials in source, tokens in the database, no rules in repo
● Medium	9	Single points of failure, duplicated logic, dev/prod drift
● Low	8	Naming inconsistencies, dead code, tooling gaps

● High severity

H1 — Gemini API key hard-coded in the n8n workflow

Where: four `HttpRequest` nodes carry the key inline in the URL query string — `Call Gemini Agent`, `Gemini Generate Content1`, `Call Gemini API1`, `Call Gemini`.

Impact: the key is written into `EVOKE-CMO-v21-GHL.json`, so it is exposed to anyone who receives a workflow export — including this handover file. It is also visible in n8n execution logs.

Fix: create an n8n credential (or use an environment variable) and reference it from all four nodes. **Rotate the key** — assume it is compromised, because the export has been shared.

H2 — Twilio Account SID hard-coded in the n8n workflow

Where: four nodes embed the Account SID directly in the Twilio Messages endpoint URL (`Day WhatsApp Send`, `Day SMS Send`, `WhatsApp Send1`, `SMS Send1`).

Fix: move to an n8n Twilio credential. The auth token appears to be handled separately — confirm before rotating.

H3 — Only 2 of 75 n8n HTTP nodes use a managed credential

Where: across the whole workflow, only `Query Due Content` and `Mark Content Published` declare `authentication: predefinedCredentialType` (`googleApi`). Every other HTTP node carries its auth inline or receives it in the request body.

Impact: no central rotation, no audit trail, secrets leak into exports and logs.

Fix: systematically migrate to n8n credentials. Start with H1 and H2.

H4 — Platform access tokens stored inside Firestore content documents

Where: `scheduleCampaignDays()` in `src/services/contentService.js` writes a `userCredentials` field onto each `content_items` document.

Why it exists: the n8n cron publishes headlessly with no user session, so it needs the tokens at publish time. This is a real constraint, not an oversight.

Impact: access tokens for Facebook, Instagram, LinkedIn, Gmail and WhatsApp are duplicated into content rows. Any read access to `content_items` is effectively token access. Tokens are also not rotated or cleared after publishing.

Fix options (in order of preference): 1. Have n8n read the tokens from `users/{uid}.socialAccounts` at publish time, using the `userId` already on the document — the credentials never need to be duplicated 2. If duplication is unavoidable, clear `userCredentials` in the `Mark Content Published` write-back 3. At minimum, ensure the Firestore rules make `content_items` readable only by its owner

H5 — Firestore security rules are not in the repository

Where: absent. Managed only in the Firebase console.

Impact: `content_items` and `governance_audit_log` are **top-level collections keyed by a `userId` field**. Collection structure alone provides no isolation — if the rules don't enforce `userId` matching on both, any authenticated user can read every user's content (and, per H4, their access tokens).

Fix: export the current rules, commit them to the repo, and review them. This should be the **first thing** the next developer does.

H6 — Secrets carrying the `VITE_` prefix

Vite inlines every `VITE_`-prefixed variable into the **client bundle**, making it publicly readable in the browser.

Variable	Actual sensitivity	Currently read from
<code>VITE_TIKTOK_CLIENT_SECRET</code>	Secret	—
<code>VITE_META_APP_SECRET</code>	Secret	—
<code>VITE_GROQ_API_KEY</code>	Secret	Server-side only (<code>process.env</code> / <code>LoadEnv</code>)
<code>VITE_FACEBOOK_PAGE_ACCESS_TOKEN</code>	Secret	—
<code>VITE_INSTAGRAM_ACCESS_TOKEN</code>	Secret	—
<code>VITE_EVOKE_API_KEY</code>	Secret	—
<code>VITE_WAVESPEED_API_KEY</code>	Secret	—

`VITE_GROQ_API_KEY` is the **least dangerous of these** — despite the prefix it is only ever read server-side, so it does not currently reach the bundle. It should still be renamed to `GROQ_API_KEY` so the prefix stops implying otherwise.

Fix: audit which of these actually reach client code. Rename every server-side one to drop the prefix, and **rotate any that are confirmed to be in a shipped bundle**.

Verify with:

```
npm run build && grep -rE "sk-|AIza|EAA" dist/assets/*.js | head
```

Medium severity

M1 — n8n is a single point of failure

Campaign generation, all social publishing, every OAuth token exchange, paid-ads creation and scheduled posting **all** depend on one self-hosted n8n instance (`n8n-zvxi.srv1837606.hstgr.cloud`). If it goes down, the product's core value proposition stops working. There is no fallback path and no health monitoring.

Fix: add uptime monitoring on the webhook endpoints at minimum. Longer term, consider moving the highest-value flows (publish, scheduler) into Vercel functions where they'd be covered by Vercel's own reliability.

M2 — The n8n host is hard-coded in two places

Location	Configurable?
<code>src/config.js</code>	✓ via <code>VITE_N8N_BASE</code>
<code>src/services/ghlService.js</code> — <code>const N8N_BASE = 'https://n8n-zvxi...'</code>	✗ hard-coded
<code>vite.config.js</code> — <code>/n8n-webhook</code> proxy target	✗ hard-coded

Changing hosts requires editing source, not just configuration.

Fix: route both through `src/config.js`.

M3 — The campaign prompt is duplicated

The 15-field campaign schema and the four-strategy `extractJson()` parser exist **twice**: `api/generate-campaign.js` (production) and `vite.config.js` (dev). They are currently identical and must be edited together — a divergence would make dev and production produce different output.

Fix: extract to a shared module both can import.

M4 — Dev and production image generation differ

	Production (<code>api/generate-banner.js</code>)	Dev (<code>vite.config.js</code>)
<code>provider: 'dalle'</code> model	<code>gpt-image-1</code>	<code>dall-e-3</code>
Size	1536×1024	1024×1024
Quality	<code>high</code>	<code>hd</code>
Return path	base64 directly (<code>b64_json</code>)	fetches the returned URL, then converts

Images generated locally will not match production.

Fix: align the two. The production path (`gpt-image-1` , direct base64) is the better one.

M5 — `/api/run-agent` exists only in dev

The nine specialist agent prompts (`reddit` , `seo` , `writer` , `twitter` , `linkedin_agent` , `hackernews` , `geo` , `coding` , `ugc_videos`) are implemented in `vite.config.js` with **no production equivalent** in `api/`. In production the equivalent work runs through the `n8n evoke-agents` webhook — **on Gemini, not Groq**.

Impact: same prompts, different models. Agent output differs between environments, and the two prompt sets can drift apart silently.

M6 — Supabase migration is half-finished

Built, then reverted on 2026-08-04. What exists:

Piece	State
<code>docs-internal/supabase-schema.sql</code>	✔ Complete, with RLS and triggers
<code>api/brand-kb.js</code> + <code>api/_lib/*</code>	✔ Written and correct
<code>src/lib/apiAuth.js</code> , <code>src/lib/supabaseClient.js</code>	✔ Present
<code>api/supabase-session.js</code>	✗ Referenced by the schema comments but does not exist. Without it <code>auth.uid()</code> never resolves and every RLS policy denies
<code>FIREBASE_ADMIN_*</code> credentials	✗ Not set
<code>SUPABASE_SERVICE_ROLE_KEY</code>	✗ Not set
Frontend wiring	✗ Reverted to Firestore

Why it failed: the API routes 404 under plain `vite` , so every save silently failed and nobody saw an error.

To reinstate: add both credential sets, **write** `api/supabase-session.js` , test under `vercel dev` or a real deployment, then repoint `knowledgeBaseService.js` .

M7 — `userPlan` and `selectedPlan` are duplicated

`saveSelectedPlan()` writes both. `useUserPlan()` reads `data?.userPlan || data?.selectedPlan || 'free'` . Two fields must be kept in sync for one value; a partial write leaves them inconsistent.

Fix: pick `userPlan` , backfill, and remove `selectedPlan` . The Supabase schema already resolves this — it has a single `plan` column.

M8 — Route/gate definitions disagree

Plan requirements are declared in **three** places that don't fully agree:

Route	App.jsx (what runs)	ROUTE_PLAN	FEATURE_PLAN
/image-3d	package-b	package-b	image_3d: package-c
/brand-governance	package-b	package-b	—
/compliance-agent	package-c	package-c	—
/hub/* routes	in App.jsx only	absent	absent
/blog-generator, /email-composer, /image-generator, /social-calendar	in App.jsx only	absent	absent

App.jsx is what actually enforces. ROUTE_PLAN is now largely decorative, and FEATURE_PLAN.image_3d actively contradicts the live gate.

Fix: make App.jsx derive its gates from ROUTE_PLAN, so there is one source of truth.

M9 — Google Ads metrics are not real

The google-ads-metrics webhook flows GAds Metrics Compute (a code node) straight into a response. **It never calls the Google Ads reporting API.** Any metrics shown are computed, not measured.

Low severity

L1 — Environment variable naming is inconsistent

Problem	Examples
Misspellings	twilo_sid, twilo_auth_token, GHL_PIT_tocken
Casing	Vite_facebook_secret vs VITE_META_APP_SECRET
Non-descriptive	bare client_id, client_secret
Duplicate pairs	VITE_FACEBOOK_PAGE_ID / VITE_FB_PAGE_ID ; VITE_INSTAGRAM_BUSINESS_ACCOUNT_ID / VITE_INSTAGRAM_ID ; VITE_META_AD_ACCOUNT_ID / META_ADS_AD_ACCOUNT_ID
Literally duplicated lines	VITE_GHL_LOCATION_ID (×2), VITE_WAVESPEED_API_KEY (×2)
Invalid line	A stray line starting For ... parses as a variable named For

L2 — No ESLint configuration

package.json defines npm run lint with --max-warnings 0, but no ESLint config file exists at the root and there is no eslintConfig key in package.json. The script cannot run as written. Source files also contain eslint-disable comments that nothing enforces.

L3 — No tests

No test framework, no test files, no CI configuration anywhere in the repository. For a 54,000-line codebase handling payments, OAuth tokens and publishing to users' real social accounts, this is the largest quality gap.

Suggested starting point: unit-test the pure logic first — planGate.js (canAccess), extractJson(), journeyHandoff.js mappers, session.js JWT handling. These are high-value, dependency-free, and cheap to cover.

L4 — No code splitting

App.jsx statically imports all 75 page components. The 4,488-line CampaignForm.jsx and the 4,392-line Landing.jsx load on every page visit, including the landing page.

Fix: React.lazy() + <Suspense> on the route definitions. This is a contained, high-impact change.

L5 — Two competing colour palettes

	Legacy (<code>index.css</code>)	Current (Figma screens)
Background	<code>#0e0c09</code> / <code>#070b17</code>	<code>#0A0A0F</code>
Card	<code>#1c1a13</code>	<code>#111118</code>
Gold	<code>#c8973e</code>	<code>#BE954A</code>
Border	<code>rgba(255, 255, 255, 0.06)</code>	<code>#2A2A3A</code>

Newer screens declare their palette as **inline local constants**, not CSS variables, so the two can't be reconciled from one place.

Fix: promote the current palette into `:root` in `index.css` and replace the inline constants.

L6 — Very large single-file components

File	Lines
<code>CampaignForm.jsx</code>	4,488
<code>Landing.jsx</code>	4,392
<code>ConnectAccounts.jsx</code>	2,273
<code>Results.jsx</code>	2,116
<code>PricingPage.jsx</code>	1,588
<code>PostContent.jsx</code>	1,307

These are difficult to review, test or hand over. `CampaignForm.jsx` in particular is the product's core flow and its single largest maintenance risk.

L7 — Dead and superseded code retained

Item	Status
<code>src/pages/Onboarding.jsx</code> (401 lines)	Route retired → redirects to <code>/brand-profile</code>
<code>src/pages/SetupPage.jsx</code> (473 lines)	Route retired → redirects to <code>/brand-profile</code>
<code>src/components/OnboardingModal.jsx</code> (684 lines)	Superseded by <code>OnboardingWizard.jsx</code>
<code>src/pages/ExecutiveReportingPage.jsx</code>	Retired 2026-07-17 (was pure mock data)
<code>/groq-api</code> Vite proxy	Kept "for backwards compat"
<code>.claude/worktrees/cmo-frontend-bug-798b7c/</code>	A full stale copy of the app inside the repo

The retirements are **well-commented** — each retired route in `App.jsx` explains why and when. That is good practice and should continue. But ~1,500 lines of superseded components remain importable.

⚠ Note that `BrandSetupModal.jsx` is **not** dead — `OnboardingWizard.jsx` imports `getRecommendations()` from it. Don't delete it without moving that function.

L8 — Duplicate image-generation endpoints

`api/gemini-image.js` is fully covered by `api/generate-banner.js?provider=gemini`. One can be removed.

Features presented in the UI without a real data source

Both are **honestly labelled in the code**, which is the right call. Preserve the labelling until real integrations land.

Feature	File	Current state
SEO Intelligence Center	<code>SeoIntelligenceCenterPage.jsx</code>	Its own header comment states no SEMrush / Ahrefs / Search Console integration exists — Domain Authority, Organic Traffic, Keywords Ranked, Backlinks and the keyword table have no data source
Social Inbox	<code>SocialInbox.jsx</code>	Shows an in-UI banner reading "Example conversations — no live LinkedIn/Instagram/Facebook/WhatsApp inbox is connected yet." Replies are persisted, and Suggested Reply is a real AI call
Google Ads metrics	n8n <code>google-ads-metrics</code>	Computed in a code node, not fetched (see M9)

Architectural constraints worth understanding before changing anything

These are **not defects** — they are deliberate decisions with reasons. Understand them before "fixing" them.

Decision	Why
The <code>evoked_user</code> cookie is non-httpOnly	So any <code>*.evokemarketplace.com</code> subdomain can render logged-in UI without an API round-trip. The code is explicit that this is for UI only — gated actions must still verify server-side
<code>evoked-cmo</code> responds before publishing	Generation + publishing takes over a minute. The browser gets an acknowledgement; results are read from Firestore afterwards
Brand KB is a nested field, not a subcollection	Avoids subcollection permission issues
<code>content_items</code> is top-level, not per-user	The n8n cron queries across all users with <code>allDescendants: true</code>
<code>getContentItems</code> sorts client-side	Avoids needing a composite Firestore index on <code>(userId, status, createdAt)</code> . Will need pagination at scale
GHL holds the platform apps	Lets users authorise GHL's Meta/LinkedIn/TikTok apps, so publishing works without Evoke completing app review for each network
<code>VITE_GHL_LOCATION_ID</code> has a hard-coded default	A missing env var previously disabled connecting silently , with no diagnosable cause. It is only a workspace identifier; the acting token lives server-side in n8n
GHL tokens live in n8n static data	Keeps the setup clear of an org policy that blocks service-account keys
<code>Post_Build_Payload</code> hunts recursively for <code>userId</code>	GHL's field naming for this value is undocumented and has changed before
Pollinations seed is <code>random() * 2147483647</code>	<code>Date.now()</code> overflows 32-bit and Pollinations rejects it
Groq 401 is remapped to 500	A bad server key must never surface to the client as a <i>user</i> auth error
<code>listConnectedPages</code> returns a <code>scoped</code> flag	On the shared workspace, auto-selecting an unscoped page would silently link another user's page
Trusted popup origin allow-list	Without it, any page could forge a social-connection event via <code>postMessage</code>
<code>scheduleCampaignDays</code> writes to Firestore	Replaced a browser <code>setTimeout</code> that cancelled the campaign when the tab closed

Recommended priority order for the next developer

Week 1 — Security 1. Export, commit and review the **Firestore security rules** (H5) — do this first 2. Rotate the Gemini key and move it to an n8n credential (H1) 3. Move the Twilio SID to a credential (H2) 4. Audit `VITE_`-prefixed secrets; verify the built bundle; rotate anything exposed (H6) 5. Decide on the `userCredentials`-in-Firestore approach (H4)

Week 2 — Stability 6. Add uptime monitoring on the n8n webhooks (M1) 7. Make `App.jsx` derive gates from `ROUTE_PLAN` (M8) 8. De-duplicate the campaign prompt (M3) 9. Align dev/prod image generation (M4)

Week 3 — Maintainability 10. Add an ESLint config so `npm run lint` runs (L2) 11. Add tests for the pure logic modules (L3) 12. Add route-level code splitting (L4) 13. Consolidate `userPlan` / `selectedPlan` (M7)

Backlog 14. Finish or formally abandon the Supabase migration (M6) 15. Break up `CampaignForm.jsx` (L6) 16. Unify the colour palettes (L5) 17. Remove dead code — carefully; check `BrandSetupModal` first (L7)

What is genuinely working well

Worth recording, so the next developer doesn't rewrite things that are already right:

- **Social publishing is live.** Facebook and Instagram post for real through GoHighLevel.
 - **Scheduled publishing works headlessly.** The 15-minute cron with its three-condition query and Firestore write-back is a sound design.
 - **The approval gate is correct.** `requiresApproval` genuinely prevents unreviewed content from auto-publishing.
 - **Retired routes are well documented.** Every redirect in `App.jsx` explains what replaced it and when. This is unusually good discipline.
 - **Failure modes are handled honestly.** Groq 401→500 remapping, Resend's 501, the GHSL "install the app first" message, and the two in-UI "this is example data" labels all favour telling the truth over hiding a gap.
 - **The `scoped` -flag and trusted-origin protections** in `ghlService.js` show real thought about multi-tenant safety on a shared workspace.
 - **Security-conscious touches throughout n8n:** the social-start token is deliberately not returned to the browser; the PIT endpoint echoes only the last 6 characters.
 - **`.gitignore` is correct** — `.env`, `.mcp.json`, `docs-internal/` and `.vercel` are all excluded. An earlier risk of internal `.docx` files being publicly served has been resolved; they now live in the git-ignored `docs-internal/`.
-

Part 9 — Complete File Index

Every file in the repository, with a one-line description. Use this to find where something lives; use 02–06 for the detail.

Excluded: `node_modules/`, `dist/`, `.git/`, and `.claude/worktrees/` (a stale full copy of the app — see Part 8 § L7).

Root

File	Purpose
<code>index.html</code>	HTML shell. Loads Inter from Google Fonts, injects the Google Ads gtag (<code>AW-18246572299</code>), mounts <code>#root</code>
<code>package.json</code>	Dependencies and scripts. <code>dev</code> pins port 3007
<code>package-lock.json</code>	Locked dependency tree — use npm
<code>vite.config.js</code>	Build config + the dev-only API middleware layer + CORS proxies (553 lines)
<code>vercel.json</code>	Build command, output dir, SPA catch-all rewrite
<code>.env</code>	All secrets and configuration. Git-ignored
<code>.gitignore</code>	Excludes <code>node_modules</code> , <code>dist</code> , <code>.env*</code> , <code>.mcp.json</code> , <code>.vercel</code> , <code>docs-internal/</code> , <code>.vscode</code> , <code>.claude</code>
<code>.mcp.json</code>	Editor tool configuration. Git-ignored (contains tokens)
<code>skills-lock.json</code>	Editor tooling lockfile

`api/` — Backend (Vercel functions)

File	Runtime	Purpose
<code>generate.js</code>	Edge	Groq chat-completions proxy — keeps the key server-side
<code>generate-campaign.js</code>	Edge	Full 15-field campaign package via Groq <code>llama-3.1-8b-instant</code> , with a four-strategy JSON extractor
<code>generate-banner.js</code>	Edge	Image generation — Gemini (default) / OpenAI <code>gpt-image-1</code> / Pollinations. Always returns base64
<code>gemini-image.js</code>	Edge	Gemini-only image generation. Overlapped by <code>generate-banner.js</code>
<code>eventbrite.js</code>	Edge	Eventbrite proxy — <code>create_venue</code> , <code>create_event</code> , <code>create_ticket</code> , <code>publish_event</code> . Caller supplies the token
<code>send-email.js</code>	Edge	Transactional email via Resend. Returns 501 until <code>RESEND_API_KEY</code> is set
<code>brand-kb.js</code>	Node	Brand KB read/write via Supabase, uid-scoped. Not in the live path
<code>_lib/supabaseAdmin.js</code>	—	Service-role Supabase client (bypasses RLS). Never import from <code>src/</code>
<code>_lib/verifyUser.js</code>	—	Returns a trusted uid: verified Firebase ID token in production, plain <code>uid</code> in dev

`src/` — Entry points & config

File	Purpose
<code>main.jsx</code>	React 18 root. StrictMode + <code>App</code> + <code>index.css</code>
<code>App.jsx</code>	Router, ~100 route definitions, the <code>G()</code> plan-gate helper, <code>EvokeAuthHandler</code> , global <code>Chatbot</code>
<code>config.js</code>	n8n webhook URL resolution with env → dev-proxy → production fallback. Documents how to switch n8n accounts

File	Purpose
<code>firebase.js</code>	Firestore init from env vars. Exports <code>auth</code> , <code>db</code> , <code>storage</code> , OAuth providers
<code>index.css</code>	The only stylesheet (347 lines). CSS custom properties, reset, base elements, scrollbar

`src/components/` — 20 shared components

File	Lines	Purpose
<code>AppSidebar.jsx</code>	372	Main navigation. Three plan-filtered groups, collapsible 260→64px, portalled account menu. Broadcasts <code>--evox-sidebar-w</code>
<code>SidebarLayout.jsx</code>	—	Wrapper: sidebar + width-offset content area
<code>Navbar.jsx</code>	444	Top nav for public pages and the plan gate
<code>ToolTopBar.jsx</code>	—	Compact header for tool pages
<code>StrategyTopBar.jsx</code>	—	Top bar variant for the strategy section
<code>JourneyFooter.jsx</code>	—	12-step CMO journey progress strip
<code>AuthProvider.jsx</code>	26	Creates <code>AuthContext</code> ; also mounts <code>OAuthCallbackHandler</code>
<code>OAuthCallbackHandler.jsx</code>	—	Detects OAuth return params and completes the connection
<code>PlanGate.jsx</code>	213	Blurred page behind a lock card with price + highlights + upgrade CTA. Bypassed on localhost
<code>UpgradeModal.jsx</code>	—	Inline upgrade prompt for gated actions
<code>OnboardingWizard.jsx</code>	588	The live 6-step onboarding flow
<code>BrandSetupModal.jsx</code>	525	Modal brand setup. ⚠️ Also exports <code>getRecommendations()</code> , imported by <code>OnboardingWizard</code>
<code>OnboardingModal.jsx</code>	684	Legacy chat onboarding. Superseded
<code>ProductTour.jsx</code>	—	First-visit guided tour anchored to sidebar <code>data-tour</code> attributes
<code>BannerGenerator.jsx</code>	452	AI banner UI — prompt, provider, preview, download
<code>EventBannerGenerator.jsx</code>	—	Event-specific banner variant
<code>Chatbot.jsx</code>	480	Floating assistant on every page
<code>ProductLaunchModal.jsx</code>	498	Guided product-launch campaign builder
<code>LoadingSpinner.jsx</code>	—	Shared loading indicator
<code>EgtWalletHeader.jsx</code>	360	EGT wallet balance in the header
<code>GratitudeToken.jsx</code>	—	EGT token display/award

`src/hooks/` — 4 hooks

File	Purpose
<code>useAuth.js</code>	Reads <code>AuthContext</code> → <code>{ profile, status, user }</code> . The hook pages should use
<code>useEvokeSession.js</code>	SSO session engine. <code>useSyncExternalStore</code> on the cookie + localhost <code>DEV_PROFILE</code> fallback
<code>useRequireAuth.js</code>	Redirects unauthenticated users to the accounts portal
<code>useUserPlan.js</code>	Plan + 14-day <code>trialDaysLeft</code> from Firestore

src/lib/ — 12 logic modules

File	Purpose
session.js (397)	Cross-domain SSO layer — cookie read/write, JWT decode, sign-out marker, backend session recovery
authUtils.js	profileToUser (uid = 'sso_' + custID) and redirectToLogin
planGate.js (177)	PLANS , FEATURE_PLAN , ROUTE_PLAN , PLAN_HIGHLIGHTS , canAccess()
journeyHandoff.js	The canonical 12-step CMO journey + step-to-step payload mappers
recommendations.jsx	getRecommendedActions() — industry-aware scored suggestions. Shared by Dashboard and Brand KB
campaignPrefill.js	Builds pre-filled campaign form values from prior context
swot.jsx	parseSwot , SwotGrid , timeAgo
apiAuth.js	authedFetch — Firebase ID token in prod, plain uid in dev
supabaseClient.js	Browser Supabase client
gtag.js	gtagEvent , trackPurchase , trackSignup
evokeUserCookie.js	Reads the EGT wallet address from the SSO cookie
egtRewardPoolBalance.js	Reads the EGT reward-pool balance via RPC
formatEgtBalance.js	EGT balance display formatting

src/services/ — 14 data-access modules

File	Purpose
userService.js	User profile, plan, tokens, socialAccounts , onboarding state. Also TOKEN_PACKAGES
contentService.js	The content_items collection — save, schedule, approve, publish. scheduleCampaignDays feeds the n8n cron
ghlService.js	GoHighLevel social publishing — location provisioning, OAuth popup, page attach, publish
knowledgeBaseService.js	Brand KB as a nested field on users/{uid} . Header documents the Supabase rollback
metaAdsService.js	Meta Ads OAuth + boost creation
googleAdsService.js	Google Ads OAuth, campaign creation, metrics
bannerService.js (353)	Banner generation — canvas composition or /api/generate-banner
crmService.js	CRM contacts + stage-lifecycle email automation (never throws)
teamService.js	Team members, roles, invitations
governanceService.js	Brand Governance audit trail
socialInboxService.js	Persists Social Inbox replies only — no live message source exists
eventService.js	Event landing-page generation and hosting
emailService.js	Thin client for /api/send-email
qrCodeService.js	QR codes via the free QR Server API

Public & legal

[Landing.jsx](#) (4,392) · [PricingPage.jsx](#) (1,588) · [PlansPage.jsx](#) (392) · [FreePlanPage.jsx](#) · [PackageAPage.jsx](#) · [PackageBPage.jsx](#) (801) · [PackageCPage.jsx](#) (665) · [Privacy.jsx](#) · [Terms.jsx](#) · [SignIn.jsx](#) (redirects to the SSO portal)

Onboarding & brand (page list)

[BrandProfilePage.jsx](#) (hosts [OnboardingWizard](#); also [?tab=settings](#)) · [BrandKnowledgeBase.jsx](#) (585) · [Onboarding.jsx](#) (401) (retired) · [SetupPage.jsx](#) (473) (retired)

Dashboard & strategy (page list)

[DashboardPage.jsx](#) (761) · [StrategyHome.jsx](#) · [MarketingStrategyPage.jsx](#) (591) · [CompetitorIntelPage.jsx](#) · [SwotAnalysisPage.jsx](#) · [MarketingHealthPage.jsx](#) (466) · [KpiRecommendationsPage.jsx](#) (639)

Campaigns (page list)

[CampaignForm.jsx](#) (4,488 — the core flow) · [Results.jsx](#) (2,116) · [CampaignsPage.jsx](#) · [NewCampaignWizardPage.jsx](#) (431) · [CampaignHub.jsx](#) · [CampaignPerformancePage.jsx](#)

Content generation (page list)

[CaptionSuitePage.jsx](#) (444) · [ReelScriptsPage.jsx](#) (604) · [ContentGenerationPage.jsx](#) (694) · [CopywritingAgentPage.jsx](#) (619) · [ProductDescription.jsx](#) (393) · [BlogGeneratorPage.jsx](#) · [EmailComposerPage.jsx](#) · [ContentStudioHubPage.jsx](#)

Creative & media (page list)

[CreativeAssetPage.jsx](#) (808) · [ImageToolPage.jsx](#) (684 — serves 5 routes) · [AIImageGeneratorPage.jsx](#) · [VideoGenerationPage.jsx](#) (641) · [VideoStudioHubPage.jsx](#) · [CreativeStudioHubPage.jsx](#) · [ProductsPage.jsx](#) (540)

Agents (page list)

[AgentsHub.jsx](#) (375) · [CmoAgentOverviewPage.jsx](#) (522) · [SeoAgentPage.jsx](#) (634) · [SeoIntelligenceCenterPage.jsx](#) (⚠️ no real SEO data source — says so in its own comment) · [EmailMarketingPage.jsx](#) (542) · [CSuitePage.jsx](#) (472 — serves all 4 C-suite routes) · [CompliancePage.jsx](#) · [BrandGovernancePage.jsx](#) (431)

Social & publishing (page list)

[ConnectAccounts.jsx](#) (2,273) · [PostContent.jsx](#) (1,307) · [ApprovalQueue.jsx](#) (635) · [SocialInbox.jsx](#) (401) (⚠️ example threads; replies are real) · [SocialMediaManagerPage.jsx](#) · [SocialCalendarPage.jsx](#) · [EventbritePost.jsx](#) (598)

Ads (page list)

[MetaAdsBoost.jsx](#) (707) · [AdsCenterHubPage.jsx](#) · [MarketingExecutionPage.jsx](#) (610)

Analytics & reporting (page list)

[AnalyticsDashboard.jsx](#) (728) · [ExecutiveReportPage.jsx](#) (548) · [MarketingAttributionPage.jsx](#) (576) · [ABTestingPage.jsx](#) (513) · [TrendAnalysis.jsx](#) (434) · [AudienceBuilder.jsx](#) (601)

Business operations (page list)

[CrmPage.jsx](#) (693) · [TeamManagement.jsx](#) (440) · [PartnerSharing.jsx](#) (308) · [Tokens.jsx](#) (872) · [Purchase.jsx](#) (loads Razorpay dynamically)

Dev-only (redirect to / in production)

[DevResetPage.jsx](#) · [SupabaseTestPage.jsx](#)

public/ — Static assets

File	Purpose
<code>evoke-logo.png</code> , <code>evoke-wordmark.svg</code> , <code>favicon.svg</code>	Branding
<code>signin-hero.webp</code>	Sign-in page hero
<code>gratitude-token.png</code>	EGT token graphic
<code>analytics-connected.png</code> , <code>analytics-platforms.png</code> , <code>analytics-tokens.png</code> , <code>analytics-top.png</code> , <code>analytics-week.png</code>	Analytics illustrations
<code>creative-hub/ai-image-generator.png</code> , <code>lifestyle-narratives.png</code> , <code>product-catalog.png</code> , <code>rotational-views.png</code> , <code>social-reels.png</code>	Creative hub tiles
<code>icons/gmail.png</code> , <code>icons/tiktok.png</code>	Platform icons
<code>tiktokGYdTycqX1no9L7SYXZqzYwbyML5WqPQf.txt</code>	TikTok domain-verification file — do not delete

docs-internal/ — Internal specs (git-ignored)

File	Purpose
<code>EVOX-CMO-Doc.docx</code>	Original product specification
<code>EVOX-CMO-Gap-Analysis.docx</code>	Gap analysis against the spec
<code>supabase-schema.sql</code>	Complete Supabase schema — tables, indexes, triggers, RLS policies

docs/ — This documentation

File	Contents
<code>README.md</code>	Index + 30-second architecture + quick start
<code>01-system-overview.md</code>	Architecture, tech stack, the complete start-to-finish journey
<code>02-frontend-reference.md</code>	Every file in <code>src/</code>
<code>03-backend-reference.md</code>	Every file in <code>api/</code> + the Vite dev API layer
<code>04-n8n-automation.md</code>	All 25 webhooks + the scheduler
<code>05-integrations.md</code>	Every third-party service
<code>06-data-model.md</code>	Firestore + Supabase schemas
<code>07-environment-and-deployment.md</code>	Env vars, local dev, deployment, troubleshooting
<code>08-known-gaps-and-risks.md</code>	Honest state assessment + priority order
<code>09-file-index.md</code>	This file

External artifacts (not in the repository)

Artifact	Location	Notes
n8n workflow	<code>EVOKE-CMO-v21-GHL.json</code>	230 nodes. Documented in 04 . Must be handed over separately

Artifact	Location	Notes
Firestore security rules	Firebase console	⚠ Not version-controlled — see Part 8 § H5
Vercel environment variables	Vercel dashboard	Must mirror <code>.env</code>
GHL Marketplace app	GoHighLevel agency account	Client id/secret + the PIT
Platform app registrations	Meta, LinkedIn, TikTok, Google, Eventbrite developer consoles	OAuth redirect URLs are registered against specific hosts

Handover checklist

Things the next owner needs that are **not** in this repository:

- [] `.env` file (or the Vercel environment variable set)
- [] `EVOKE-CMO-v21-GHL.json` — the n8n workflow export
- [] n8n instance access (`n8n-zvxi.srv1837606.hstgr.cloud`)
- [] Firebase console access — project `evoke-cmo-agent2`
- [] **The current Firestore security rules** (export and commit them)
- [] Vercel project access
- [] GoHighLevel agency account access
- [] Developer console access: Meta, LinkedIn, TikTok, Google Ads, Eventbrite
- [] Groq, Gemini, OpenAI, ImgBB, Resend, Cloudinary, Razorpay accounts
- [] Supabase project access (if the migration is to be finished)